

Instituto Tecnológico y de Estudios Superiores de Monterrey
Escuela de Ingeniería y Ciencias

Analizador Sintáctico

Triton GPU Kernel

TC3002B — Computer Science Advanced Applications Development
Módulo #3: Compiladores — Fase de Análisis Sintáctico

Equipo: 11

Cristóbal Camarena Hernández — A01642653

Josué Galindo Gutiérrez — A01637983

Gandhi Emmanuel Valdez Huerta — A01625738

Diego Alberto Estrada López — A01638657

Profesor: Dr. Salvador Hinojosa

Fecha de entrega: Junio 2026

Guadalajara, Jalisco, México

Resumen

Resultados: el analizador sintáctico `triton_parser` obtiene **12/12 PASS** en la suite unitaria, **200/200 (100 %)** en las suites JSONL (`curated_100` + `adversarial_100`) y **58/70 PARSE_OK (82.9 %)** en el benchmark TRITONHEAL. La métrica principal de validación es JSONL: **100 %** de kernels válidos aceptados y **100 %** de casos inválidos rechazados, sin falsos positivos ni falsos negativos. En el benchmark, 2 rechazos son mutaciones sintácticas sembradas; 10 adicionales corresponden a archivos con código suelto en columna 0 (inválido en Python). Este reporte documenta la CFG, el diseño yacc/Flex, las tablas de símbolos, los mensajes de error y la evidencia experimental.

Índice

1. Introducción	1
1.1. Resumen	1
1.2. Notación	1
1.3. Justificación de herramientas	1
1.4. Trazabilidad a la fase léxica	1
2. Análisis	1
2.1. Entregables del reto (Sección IV, documento de evaluación)	1
2.2. Requerimientos funcionales	2
2.3. CFG (resumen)	2
2.4. Pasos para obtener la gramática	3
2.5. Simplificaciones justificadas	3
2.6. Modificaciones al analizador léxico	4
2.7. Patrones léxicos: <code>BAD_ID</code>	5
2.8. Trazabilidad token léxico $\rightarrow$ yacc	5
3. Diseño	5
3.1. Arquitectura del sistema	5
3.2. Integración <code>lex</code> $\leftrightarrow$ <code>yacc</code>	5
3.3. Diseño de la especificación yacc	6
3.4. Tabla de símbolos del parser (diseño)	7
3.5. Mensajes de error (diseño)	7
3.6. Trazabilidad diseño $\rightarrow$ análisis	7
4. Implementación	7
4.1. Visión general	7
4.2. Estructura del proyecto	7
4.3. Compilación y ejecución	8
4.4. Código fuente completo y comentado	8
4.4.1. Archivo: <code>parser/symtab.h</code>	8
4.4.2. Archivo: <code>parser/symtab.c</code>	10
4.4.3. Archivo: <code>src/driver.c</code>	14
4.4.4. Archivo: <code>parser/parser.y</code>	17
4.5. Ejemplo de salida del parser (Entregable 5)	29
4.6. Mensajes de error (Entregable 4)	30
4.7. Explicación completa de las secciones de Flex (integración con yacc)	30
4.7.1. Sección 1 — Declaraciones de Flex (<code>%{... %}</code> y definiciones)	30
4.7.2. Sección 2 — Reglas y patrones (<code>%% ... %%</code>)	31
4.7.3. Sección 3 — Código C auxiliar (<code>%% final</code>)	32

5. Verificación y Validación	33
5.1. Resultados experimentales	33
5.2. Resumen cuantitativo	34
5.3. Suites JSONL (<code>make jsonl-test</code>)	34
5.4. Interpretación de los resultados	34
5.5. Gráfica de resultados (Figura 2)	35
5.6. Detalle de rechazos en el benchmark	37
5.7. Modelo de pruebas	37
5.8. Suite unitaria (<code>make test</code>)	37
5.9. Evidencia: salida exitosa (<code>ejemplo_1.tri</code>)	37
5.10. Evidencia: error sintáctico (<code>def_sin_dos_puntos.tri</code>)	37
5.11. Evidencia: error léxico <code>BAD_ID</code> (I6)	37
5.12. Reproducibilidad	37
5.12.1. Prerrequisitos	38
5.12.2. Pasos desde cero (PC nueva)	38
5.12.3. Comando único (regeneración completa)	38
5.12.4. Salidas verificables	38
5.12.5. Consulta sin recompilar	39
5.12.6. Notas por sistema operativo	39
6. Plan de Trabajo	39
6.1. Distribución de responsabilidades	39
A. Gramática libre de contexto completa	39
A.1. Convenciones y notación	39
A.2. Símbolo inicial y conteo	40
A.3. Terminales del analizador	40
A.3.1. Terminales que aparecen en producciones	40
A.3.2. Terminales declarados y no usados en reglas	40
A.4. Precedencia y asociatividad (<code>yacc</code>)	41
A.5. Inventario de no terminales (41)	41
A.6. Notas semánticas ligadas a la CFG (fuera de producciones)	42
A.7. Desglose detallado: no terminal <code>expr</code> (16 alternativas, P-065–080)	42
A.8. Desglose detallado: <code>subscript</code> (5 alternativas, P-085–089)	43
A.9. Desglose detallado: <code>block_stmt</code> (2 alternativas, P-014–015)	43
A.10. Listado completo de producciones (P-001 a P-105)	43
A.11. Implementación <code>yacc</code> (referencia cruzada)	46
A.12. Trazabilidad y mantenimiento	46
B. Referencias	46

Índice de figuras

1. Pipeline del analizador sintáctico.	5
2. Resultados del analizador sintáctico (benchmark 70 kernels). Verde = <code>PARSE_OK</code> ; naranja/rojo = rechazo (mutaciones <code>k_0040/k_0058</code> o código en columna 0). La validación JSONL 200/200 no se grafica aquí; ver Tabla 8.	36

Índice de cuadros

1.	Producciones épsilon en la CFG y su justificación.	3
2.	Resumen de simplificaciones aplicadas a la CFG.	4
3.	Caracteres inválidos en identificadores y patrón Flex.	5
4.	Correspondencia Token ID (reporte léxico) y yacc.	5
5.	Esquema de entradas en <code>syntab.c</code>	7
6.	Archivos fuente del analizador sintáctico y su función.	7
7.	Cuadro resumen de resultados (corridas locales reproducibles con <code>make test</code> , <code>make jsonl-test</code> y <code>make benchmark</code>).	34
8.	Resultados JSONL (métrica principal del proyecto).	34
9.	Clasificación de los 12 rechazos del benchmark TRITONHEAL.	37
10.	Casos de prueba del equipo (12 ejecuciones).	37
11.	Software necesario para reproducir compilación, pruebas, gráfica e informe.	38
12.	Cronograma de la fase sintáctica (Equipo 11).	39
13.	Resumen formal de la gramática implementada.	40
14.	Terminales usados en al menos una producción de la CFG.	40
15.	Directivas <code>%left</code> / <code>%nonassoc</code> / <code>%right</code> en <code>parser.y</code> (L71–78).	41
16.	Catálogo de no terminales: número de alternativas e identificadores P- en <code>triton_cfg.bnf</code>	41
17.	Acciones en C asociadas a reglas seleccionadas (no son parte de la CFG).	42

1. Introducción

1.1. Resumen

Este documento presenta el análisis, diseño, implementación y verificación del **analizador sintáctico** para kernels **Triton GPU** escritos en archivos `.tri`. La fase sintáctica es la segunda etapa del compilador del reto académico: consume la secuencia de tokens producida por el analizador léxico (Flex) del Equipo 11, construye una estructura conforme a una gramática libre de contexto (CFG) implementada con **yacc/Bison**, actualiza tablas de símbolos y reporta errores sintácticos.

El entregable incluye: (i) gramática documentada (Apéndice A, CFG completa); (ii) tablas de símbolos; (iii) implementación `triton_parser`; (iv) mensajes de error; (v) **resultados experimentales** con énfasis en la Sección 5.1: **100 %** suite unitaria (12/12), **100 %** suites JSONL (200/200), benchmark TRITONHEAL (58/70) y gráfica de los 70 kernels.

1.2. Notación

- **CFG**: gramática libre de contexto en notación BNF; producciones $A \rightarrow \alpha$.
- **Token ID**: identificador numérico del Cuadro 1 del reporte léxico (100–999).
- **INDENT/DEDENT** (910/911): tokens sintéticos derivados de `WS_COUNT` (900) para modelar indentación Python.
- **PARSE_OK**: análisis sintáctico exitoso con impresión de tabla de símbolos del parser.

1.3. Justificación de herramientas

Se utiliza **C** con **Flex** y **yacc/Bison**, conforme a las restricciones del curso [1]. No se emplean librerías CFG de alto nivel ni `ast` de Python para el parser formal. El modelo de desarrollo sigue el proceso de análisis, diseño, implementación y verificación definido en el IEEE-830 [4], con los cinco entregables del Entregable B como unidades de documentación.

1.4. Trazabilidad a la fase léxica

Los IDs de token se preservan en `%token` de `parser.y` (p. ej. `TOKEN_IDENTIFIER = 100`). El archivo `lexico_equipo_11_scanner.l` es la base del scanner integrado; el modo `-t` reproduce la salida del reporte léxico (TC-01–TC-04).

2. Análisis

2.1. Entregables del reto (Sección IV, documento de evaluación)

1. Gramática correcta y no ambigua para kernels Triton, con explicación de decisiones.
2. Gestión de tablas de símbolos durante el análisis sintáctico.
3. Implementación del parser con yacc.
4. Mensajes de error sintácticos.
5. Ejemplos de salida del parser.

2.2. Requerimientos funcionales

- Reconocer programas `.tri`: `imports`, `@triton.jit`, `def`, cuerpos indentados.
- Sentencias: asignación (`ident_stmt`, anotada `id : tipo =`), `if/elif/else`, `while`, `for/in`, `return`, `pass`.
- Expresiones: literales, operadores (500), operador `is`, expresión condicional (`expr if expr else expr`), llamadas `tl.*`, subíndices `[: , None]`, argumentos con nombre (`mask=`, `dtype=`).
- Validación de línea: rechazo de múltiples sentencias en la misma línea (`lexer + validate_stmt_line_end`).
- Indentación: DEDENT automático cuando una línea comienza en columna 0 tras un bloque indentado.
- Rechazar sintaxis inválida con `SYNTAX_ERROR` en español.
- Actualizar `syntab`: `imports`, funciones, parámetros, variables.

2.3. CFG (resumen)

La gramática formal completa (**41 no terminales**, **105 alternativas** numeradas P-001...P-105, inventario en Tabla 16 del Apéndice A) se documenta en el Listado 10 y el bloque yacc Listado 5. El siguiente extracto resume las construcciones principales para lectura rápida en el análisis.

Listing 1: CFG resumida del lenguaje `.tri` (Triton kernel subset)

```
1 program      -> opt_newlines top_block opt_newlines
2 top_block    -> top_block top_line | epsilon
3 top_line     -> stmt | stmt NEWLINE
4
5 stmt         -> import_stmt | def_stmt | assign_stmt | ident_stmt | if_stmt
6               | while_stmt | for_stmt | return_stmt | pass_stmt | expr_stmt
7
8 ident_stmt   -> TOKEN_IDENTIFIER ident_op
9 ident_op     -> TOKEN_ASSIGN expr | ident_suffix
10
11 block_stmt  -> stmt TOKEN_NEWLINE | stmt /* validate_stmt_line_end */
12
13 import_stmt  -> KW_IMPORT import_name opt_as
14 import_name -> ID | import_name '.' ID
15
16 opt_triton_decorator -> epsilon | '@' ID '.' ID opt_newlines
17
18 def_stmt     -> opt_triton_decorator KW_DEF ID '(' param_list_opt ')' ':' suite
19
20 suite        -> simple_stmt
21               | opt_newlines INDENT block_body DEDENT
22
23 expr         -> literal | ID ident_suffix
24               | expr OP expr | expr KW_IS expr
25               | expr KW_IF expr KW_ELSE expr
26               | expr '[' subscript_list ']'
27               | '(' arg_list_opt ')' | call_expr | unary_expr
28
29 subscript    -> expr | NONE | ':' opt_expr | opt_expr ':' opt_expr
30               | opt_expr ':' opt_expr ':' opt_expr
31
32 call_expr    -> ID '(' arg_list_opt ')'
33               | ID member_chain '(' arg_list_opt ')'
34
35 arg          -> expr | ID '=' expr
```

2.4. Pasos para obtener la gramática

La metodología de análisis sigue el modelo descrito en [1] para la fase sintáctica, y la construcción de la CFG se basa en los principios del Capítulo 3 del compendio de la asignatura [1], así como en los fundamentos teóricos de Aho et al. [6].

1. Inventario de tokens del reporte léxico (Cuadro 1); trazabilidad numérica preservada [3].
2. Identificación de estructuras Triton en 70 kernels de referencia del benchmark TRITONHEAL [7].
3. Borrador de producciones para bloques indentados (`INDENT/DEDENT`).
4. Incorporación de expresiones, llamadas y subíndices tensoriales propios del lenguaje Triton [5].
5. Resolución de ambigüedades con precedencias yacc y pruebas iterativas contra suites JSONL.

2.5. Simplificaciones justificadas

Las siguientes simplificaciones se aplicaron durante el diseño de la CFG para eliminar ambigüedades, reducir conflictos shift/reduce y mantener la gramática dentro de las capacidades LALR(1) de yacc/Bison.

1. Producciones unitarias. Una producción unitaria tiene la forma $A \rightarrow B$ donde B es un único no-terminal. Una jerarquía clásica de precedencia de expresiones (`expr` $\rightarrow$ `or_expr` $\rightarrow$ `and_expr` $\rightarrow$... $\rightarrow$ `primary`) introduce múltiples producciones unitarias que degradan el rendimiento y complican la lectura. Decisión de diseño: se colapsaron todos los niveles de expresión en un único no-terminal `expr`, y la precedencia se delega a las directivas `%left`/`%nonassoc`/`%right` de yacc, que son más expresivas y directas. **Impacto:** eliminación de ≈ 6 no-terminales intermedios (`or_expr`, `and_expr`, `not_expr`, `comp_expr`, `add_expr`, `mul_expr`) y sus producciones unitarias correspondientes.

2. Producciones épsilon (ϵ). Las producciones $A \rightarrow \epsilon$ se conservan solo donde el lenguaje lo requiere semánticamente; en ningún caso se añadieron por conveniencia:

Cuadro 1: Producciones épsilon en la CFG y su justificación.

No-terminal	Justificación de la producción vacía
<code>opt_newlines</code>	Las líneas en blanco entre declaraciones son opcionales en Python.
<code>top_block</code>	Un archivo puede estar vacío (solo imports o vacío).
<code>param_list_opt</code>	Una función puede tener cero parámetros: <code>def f():</code> .
<code>block_body</code>	Un bloque indentado puede contener solo <code>pass</code> .
<code>opt_expr</code>	Los extremos de un <i>slice</i> son opcionales: <code>a[:n]</code> , <code>a[m:]</code> .
<code>elif_parts</code>	El <code>if</code> puede no tener cláusulas <code>elif</code> .
<code>else_part</code>	El <code>if</code> puede no tener cláusula <code>else</code> .
<code>opt_triton_decorator</code>	El decorador <code>@triton.jit</code> es opcional.
<code>opt_as</code>	La cláusula <code>as alias</code> en <code>import</code> es opcional.
<code>arg_list_opt</code>	Llamadas sin argumentos: <code>f()</code> .
<code>ident_suffix</code>	Un identificador puede aparecer solo, sin sufijo.

Se evitó la ambigüedad inducida por ϵ asegurando que en cada uso el conjunto FIRST del no-terminal siguiente sea disjunto con el conjunto FOLLOW del no-terminal con producción vacía, o que el conflicto sea resoluble por la directiva de precedencia correspondiente.

3. Símbolos inútiles. Un símbolo es *inútil* si es inaccesible desde el símbolo inicial (*unreachable*) o si no puede derivar ninguna cadena de terminales (*unproductive*). Se identificaron y excluyeron las siguientes construcciones de Python que *no* aparecen en los kernels Triton del subconjunto del curso:

- **class** — Triton no define clases; los kernels son funciones puras.
- **try/except/finally** — El manejo de excepciones no está dentro del subconjunto Triton del reto.
- **with / as** (gestor de contexto) — Sin instancias en los 70 kernels de referencia ni en las 100 muestras curadas.
- **yield / lambda** — Python avanzado; ningún kernel Triton del reto los usa.
- **switch** — No existe en Python; solo aparece en la tabla de palabras reservadas heredada del análisis léxico para compatibilidad.
- **del / global / nonlocal / assert** — Fuera del subconjunto definido por los entregables del reto.

Incluir reglas para estos símbolos habría añadido producciones *unreachable* en la CFG final. Su exclusión reduce el número de reglas y evita conflictos shift/reduce adicionales. Las palabras reservadas correspondientes siguen reconociéndose léxicamente (se mapean a `TOKEN_KEYWORD`), pero al no existir una producción que las use, el parser las rechaza con `SYNTAX_ERROR` si aparecen en el código de entrada, lo que es el comportamiento correcto.

Cuadro 2: Resumen de simplificaciones aplicadas a la CFG.

Tipo	Decisión	Resultado
Producciones unitarias	Colapsar jerarquía en <code>expr</code>	−6 no-terminales; precedencias con <code>%left</code>
Producciones ϵ	Solo donde requeridas semánticamente	11 producciones vacías justificadas
Símbolos inútiles	Excluir construcciones sin uso en Triton	−9 no-terminales potenciales
Indentación	<code>WS_COUNT</code> → pila → <code>INDENT/DEDENT</code>	Indentación Python como tokens sintéticos

2.6. Modificaciones al analizador léxico

- Integración con `y.tab.h`; `yylex()` retorna códigos de token.
- Tokens sintéticos 910/911 (no en Cuadro 1 original; documentados en este reporte).
- Sin nuevos IDs en el Cuadro 1 oficial; trazabilidad numérica conservada.
- **Detección `BAD_ID`** (heredada del analizador léxico del equipo en <https://github.com/G4ndhiV/Analizador-lexico>): identificadores que contienen caracteres prohibidos (`$`, acento grave, barra invertida, `?`, `!`) se reconocen con una regla Flex dedicada *antes* que `ID`, emitiendo `LEXICAL_ERROR` (999) sin fragmentar el lexema.
- **Múltiples sentencias en la misma línea:** el wrapper `yylex()` detecta patrones inválidos (`pass pass, return 5 6, x = 5 y = 6, etc.`) y marca `syntax_error_seen`.

- **DEDENT en columna 0:** si un identificador inicia una línea sin espacios previos y la pila de indentación está activa, se emite `TOKEN_DEDENT` antes del token (cierre de bloque Python).

2.7. Patrones léxicos: BAD_ID

Cuadro 3: Caracteres inválidos en identificadores y patrón Flex.

Definición	Regex	Comportamiento
BAD_ID_CHAR	<code>[\$'\?!]</code>	Conjunto de símbolos no permitidos en nombres.
BAD_ID	ver <code>lexer/lexico_equipo_11_scanner.1</code>	Secuencias con al menos un carácter inválido; prioridad sobre ID.
ID	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	Identificadores válidos (tabla 100).

Mensaje emitido: `LEXICAL_ERROR: ... (identificador con caracteres invalidos)`. El driver marca `lexical_error_seen` y termina con código de salida 2, sin reportar `PARSE_OK`.

2.8. Trazabilidad token léxico $\rightarrow$ yacc

Cuadro 4: Correspondencia Token ID (reporte léxico) y yacc.

ID	Nombre	yacc
100	IDENTIFIER	<code>TOKEN_IDENTIFIER</code>
101	KEYWORD	<code>KW_def</code> , <code>KW_if</code> , ...
200–400	Literales	<code>TOKEN_INTEGER</code> , etc.
500	OPERATOR	<code>TOKEN_OPERATOR</code>
600	DELIMITER	<code>TOKEN_DELIMITER</code> o literales <code>() [] : ,</code>
900	WS_COUNT	convertido a <code>INDENT/DEDENT</code>
950	NEWLINE	<code>TOKEN_NEWLINE</code>
910/911	(sintéticos)	<code>TOKEN_INDENT</code> , <code>TOKEN_DEDENT</code>

3. Diseño

3.1. Arquitectura del sistema

El diseño sigue el modelo en capas del compilador: **entrada** $\rightarrow$ **scanner** $\rightarrow$ **parser** $\rightarrow$ **acciones semánticas** (`syntab` + salida).

`.tri` $\rightarrow$ Flex (`lexer/`) $\rightarrow$ yacc (`parser.y`) $\rightarrow$ Syntab + stdout/stderr

Figura 1: Pipeline del analizador sintáctico.

3.2. Integración `lex` $\leftrightarrow$ `yacc`

1. Bison genera `y.tab.c/h` con códigos de token y tipos `YYSTYPE`.
2. Flex incluye `y.tab.h`; `yyflexlex()` reconoce patrones; `yylex()` inyecta `INDENT/DEDENT`.
3. `yylval.i` transporta índices de tabla léxica; `yylval.s` transporta lexemas de operadores y delimitadores.

4. `driver.c` invoca `yyparse()` y, si no hay errores, imprime `PARSE_OK` y tablas.

3.3. Diseño de la especificación yacc

La especificación `parser.y` sigue la estructura de tres secciones de yacc/Bison: preámbulo C, declaraciones y reglas. El diseño de cada sección se describe a continuación.

Preámbulo C (%{... %}). Contiene las variables de contexto compartidas entre reglas (`current_func`, `param_pos_counter`, `triton_decorator_pending`), las funciones auxiliares de recuperación de lexemas (`id_text`, `int_text`, `float_text`, `str_text`) y el prototipo de `validate_stmt_line_end`. Este diseño centraliza el estado del parser en variables estáticas de módulo, evitando el uso de un árbol sintáctico en memoria.

Sección de declaraciones.

- **%union:** define los dos tipos semánticos (`int i` para índices en tablas léxicas; `char *s` para lexemas de operadores y delimitadores).
- **%token:** los IDs numéricos coinciden exactamente con el Cuadro 1 del reporte léxico para garantizar trazabilidad.
- **Precedencias:** de menor a mayor prioridad: `TOKEN_NEWLINE < KW_OR < KW_AND < KW_NOT < TOKEN_OPERATOR < KW_ELIF < KW_ELSE < KW_IF` (expresión ternaria). Resuelven los conflictos `shift/reduce` de operadores y del *dangling-else*.

Regla inicial. `program` acepta newlines opcionales al inicio y al final, y un `top_block` que contiene imports y definiciones de función. Esta estructura refleja el nivel 0 de indentación de Python.

Bloques indentados (suite). El no-terminal `suite` unifica el cuerpo de funciones, ramas `if/elif/else`, bucles `while/for`: puede ser una sentencia simple (en la misma línea del `:`) o un bloque delimitado por `INDENT...DEDENT` emitidos por el scanner. Esto traslada la complejidad de la indentación Python al lexer, manteniendo la gramática libre de contexto.

Decorador @triton.jit. La regla `triton_decorator` verifica que la forma sea exactamente `@triton.jit`; otros decoradores se aceptan léxicamente pero el flag `triton_decorator_pending` no se activa. El campo `is_triton_jit` de `FuncEntry` registra esta información para la tabla de símbolos.

Expresiones y llamadas Triton.

- **member_chain:** modela accesos de atributo encadenados como `tl.load`, `tl.store`, `tl.math.exp`.
- **subscript_list** con **subscript:** soporta las formas de indexación tensorial `a[i]`, `a[None, :]`, `a[lo:hi:step]`.
- **arg:** incluye argumentos con nombre (`mask=m`, `dtype=tl.float16`) como los usa `tl.load`/`tl.store`.

Validación de múltiples sentencias. La función `validate_stmt_line_end()` usa `yypeek()` para inspeccionar el token siguiente sin consumirlo; si ese token puede iniciar una sentencia en la misma línea, emite `SYNTAX_ERROR`. Esta lógica complementa la detección léxica en `lexer_multistmt_error()` para cubrir todos los patrones del adversarial-100.

Mensajes de error. `yyerror()` genera mensajes en el formato estándar del curso: `SYNTAX_ERROR: line=n near="" message='msg'`. El campo `parse.error verbose` de Bison proporciona adicionalmente el token inesperado y los tokens esperados.

3.4. Tabla de símbolos del parser (diseño)

Cuadro 5: Esquema de entradas en `syntab.c`.

Tipo	Campos	Inserción
Import	módulo, alias, línea	<code>import_stmt</code>
Función	nombre, línea, aridad, <code>triton_jit</code>	cierre de <code>def_stmt</code>
Parámetro	función, nombre, posición, anotación	cada <code>param</code>
Variable	nombre, ámbito, línea	<code>assign_stmt</code> con <code>=</code>

3.5. Mensajes de error (diseño)

- **Léxico:** `LEXICAL_ERROR: token_id=<999>line=<n>lexeme="..."`.
- **Sintáctico:** `SYNTAX_ERROR: line=<n>near="" message='...'` vía `yyerror()`.

3.6. Trazabilidad diseño → análisis

Cada requisito del análisis (Sección 2) se implementa en un módulo: CFG en `parser.y`, indentación en `lexer/`, `syntab` en `syntab.c`, errores en `yyerror`, salidas en `driver.c`.

4. Implementación

4.1. Visión general

La implementación del analizador sintáctico se compone de cuatro archivos fuente en C y dos archivos de especificación de herramientas (Flex y yacc/Bison), todos ellos completamente comentados conforme al estándar de documentación del curso.

Cuadro 6: Archivos fuente del analizador sintáctico y su función.

Archivo	Herramienta	Función
<code>parser/parser.y</code>	yacc/Bison	CFG, acciones semánticas, gestión de errores
<code>parser/syntab.c</code>	C	Implementación de la tabla de símbolos del parser
<code>parser/syntab.h</code>	C	Interfaz (tipos y prototipos) de <code>syntab.c</code>
<code>src/driver.c</code>	C	Punto de entrada; modos <code>-t</code> y parser completo
<code>include/tokens.h</code>	C	Constantes numéricas de tokens compartidas
<code>lexer/lexico_equipo_11_scanner.l</code>	Flex	Scanner; genera <code>INDENT/DEDENT</code>

4.2. Estructura del proyecto

`analizador-sintactico/`

```

include/tokens.h          -- IDs de token documentados (Cuadro 1)
lexer/lexico_equipo_11_scanner.l -- Scanner Flex con indentacion Python
lexer/lexer.h             -- API publica del scanner
parser/parser.y           -- CFG + acciones semanticas (yacc/Bison)
parser/symtab.c / symtab.h -- Tabla de simbolos del parser
src/driver.c              -- Driver: modos -t (lexer) y parser
build/triton_parser       -- Ejecutable final
tests/{valid,invalid}/    -- Casos de prueba del equipo
results/{parser,jsonl,benchmark}/ -- Resultados y logs

```

4.3. Compilación y ejecución

```

git clone https://github.com/G4ndhiV/Analizador-sint-ctico.git
cd Analizador-sint-ctico
make build          # compila triton_parser
make test           # 12 casos unitarios
make jsonl-test     # 200 casos JSONL (curated + adversarial)
make benchmark      # 70 kernels benchmark/kernels/
./build/triton_parser archivo.tri      # modo parser
./build/triton_parser -t archivo.tri   # modo solo lexico

```

4.4. Código fuente completo y comentado

En esta sección se presenta el código fuente completo del analizador sintáctico. Cada archivo incluye:

1. Cabecera de archivo con propósito, descripción y relación con otros módulos.
2. Comentario de cada función/sección explicando el *qué*, *por qué* e invariantes no evidentes.
3. Comentarios inline en la lógica no trivial (manejo de indentación, desambiguación de operadores, diferimiento de tokens).

4.4.1. Archivo: parser/symtab.h

Este archivo define los cuatro tipos de entrada de la tabla de símbolos (`FuncEntry`, `ParamEntry`, `VarEntry`, `ImportEntry`) y declara la API que las acciones semánticas de `parser.y` invocan durante el análisis. La trazabilidad directa entre los entregables del análisis (funciones, parámetros, variables, imports) y los *structs* de este archivo satisface el Entregable 2 del reto.

Listing 2: `parser/symtab.h`: tipos e interfaz de la tabla de símbolos.

```

1  /*
2  * Archivo : parser/symtab.h
3  * Proposito: Interfaz de la tabla de simbolos del analizador sintactico.
4  *           Define los tipos de entrada (funcion, parametro, variable,
5  *           importacion) y declara las funciones de insercion y consulta
6  *           que las acciones semanticas de parser.y invocan durante el
7  *           analisis de cada construccion del lenguaje .tri.
8  * Relacion : Incluido por parser/parser.y (acciones semanticas) y por
9  *           parser/symtab.c (implementacion). La informacion recopilada
10 *           aqui se imprime al final de un parse exitoso mediante
11 *           symtab_print(), llamada desde src/driver.c.
12 */
13
14 #ifndef SYMTAB_H
15 #define SYMTAB_H
16

```

```

17  /*
18  * FuncEntry: registro de una funcion definida con 'def'.
19  *   name      -- nombre del identificador de la funcion.
20  *   line      -- linea del archivo fuente donde aparece 'def'.
21  *   arity     -- numero de parametros formales.
22  *   is_triton_jit -- 1 si la funcion esta decorada con @triton.jit, 0 si no.
23  */
24  typedef struct {
25      char name[256];
26      int line;
27      int arity;
28      int is_triton_jit;
29  } FuncEntry;
30
31  /*
32  * ParamEntry: registro de un parametro formal de una funcion.
33  *   func      -- nombre de la funcion a la que pertenece el parametro.
34  *   name      -- nombre del parametro.
35  *   position  -- posicion ordinal (0-based) en la lista de parametros.
36  *   annot    -- anotacion de tipo si existe (p.ej. "tl.constexpr"), vacio si no
37  *   line      -- linea donde aparece el parametro en la definicion.
38  */
39  typedef struct {
40      char func[256];
41      char name[256];
42      int position;
43      char annot[256];
44      int line;
45  } ParamEntry;
46
47  /*
48  * VarEntry: registro de una variable asignada dentro de una funcion o global.
49  *   name      -- nombre de la variable.
50  *   line      -- linea de la primera asignacion.
51  *   scope     -- nombre de la funcion que contiene la variable, o "global".
52  */
53  typedef struct {
54      char name[256];
55      int line;
56      char scope[64];
57  } VarEntry;
58
59  /*
60  * ImportEntry: registro de una sentencia 'import' o 'import ... as ...'.
61  *   module    -- nombre completo del modulo (puede ser "triton.language").
62  *   alias     -- alias introducido con 'as' (vacio si no hay).
63  *   line      -- linea del import en el archivo fuente.
64  */
65  typedef struct {
66      char module[256];
67      char alias[256];
68      int line;
69  } ImportEntry;
70
71  /* -- API de la tabla de simbolos -----
72  */
73  /* symtab_reset: vacia todas las tablas; debe llamarse antes de cada parse. */
74  void symtab_reset(void);
75
76  /* symtab_add_import: registra una sentencia import con su modulo y alias. */
77  void symtab_add_import(const char *module, const char *alias, int line);

```

```

78
79 /*
80  * syntab_add_function: registra una funcion nueva y establece el ambito
81  * actual (current_scope) a su nombre para que los parametros y variables
82  * subsiguientes se asocien correctamente.
83  */
84 void syntab_add_function(const char *name, int line, int arity, int
      is_triton_jit);
85
86 /* syntab_add_param: anade un parametro a la funcion actualmente en scope. */
87 void syntab_add_param(const char *func, const char *name, int pos,
88                      const char *annot, int line);
89
90 /*
91  * syntab_add_variable: inserta una variable si no existe ya en el mismo
92  * scope; las redeclaraciones (asignaciones repetidas) se ignoran.
93  */
94 void syntab_add_variable(const char *name, int line, const char *scope);
95
96 /* syntab_set_scope: cambia manualmente el ambito activo (uso interno). */
97 void syntab_set_scope(const char *scope);
98
99 /* syntab_print: imprime todas las entradas de las cuatro tablas por stdout. */
100 void syntab_print(void);
101
102 #endif /* SYMTAB_H */

```

4.4.2. Archivo: parser/syntab.c

`syntab.c` implementa las cuatro tablas estáticas de la tabla de símbolos del parser. Cada función de inserción es invocada desde una acción semántica específica de `parser.y`:

- `syntab_add_import()` ← acción de `import_stmt`.
- `syntab_add_function()` ← acción de `def_stmt` (tras el `':'`).
- `syntab_add_param()` ← acción de `param`.
- `syntab_add_variable()` ← acción de `assign_stmt` e `ident_stmt`.

La función `copy_str()` garantiza que ningún campo de los *structs* desborde su tamaño fijo. La detección de duplicados en `syntab_add_variable()` asegura que la misma variable no aparezca dos veces en la misma tabla de símbolos.

Listing 3: `parser/syntab.c`: implementación de la tabla de símbolos.

```

1  /*
2  * Archivo : parser/syntab.c
3  * Proposito: Implementacion de la tabla de simbolos del analizador sintactico.
4  *           Mantiene cuatro arreglos estaticos para almacenar registros de
5  *           imports, funciones, parametros y variables descubiertos durante
6  *           el recorrido de la gramatica en parser.y. Las entradas se insertan
7  *           desde las acciones semanticas de yacc (reglas import_stmt,
8  *           def_stmt, param y assign_stmt/ident_stmt).
9  * Relacion : Implementa la interfaz declarada en parser/syntab.h.
10 *           Es compilado junto con parser.y y src/driver.c para formar el
11 *           ejecutable build/triton_parser.
12 */
13
14 #include "syntab.h"
15
16 #include <stdio.h>

```

```

17 #include <string.h>
18
19 /* Capacidades maximas de cada tabla (ajustables sin cambiar la interfaz). */
20 #define MAX_FUNCS 256 /* Mazimo de funciones por archivo .tri
    */
21 #define MAX_PARAMS 1024 /* Mazimo de parametros acumulados de todas las func
    */
22 #define MAX_VARS 1024 /* Mazimo de variables (una entrada por nombre+scope)
    */
23 #define MAX_IMPORTS 64 /* Mazimo de sentencias import por archivo
    */
24
25 /* -- Arreglos estaticos de las tablas -----
    */
26 static FuncEntry funcs[MAX_FUNCS];
27 static int func_count;
28
29 static ParamEntry params[MAX_PARAMS];
30 static int param_count;
31
32 static VarEntry vars[MAX_VARS];
33 static int var_count;
34
35 static ImportEntry imports[MAX_IMPORTS];
36 static int import_count;
37
38 /* Ambito activo: nombre de la ultima funcion procesada o "global". */
39 * Se actualiza al entrar a un def_stmt y se usa como scope de las variables.
    */
40 static char current_scope[64] = "global";
41
42 /*
43  * copy_str: copia src en dst con longitud maxima n, garantizando terminacion
44  * con '\0'. Maneja src == NULL copiando una cadena vacia.
45  */
46 static void copy_str(char *dst, size_t n, const char *src) {
47     if (!src) {
48         dst[0] = '\0';
49         return;
50     }
51     strncpy(dst, src, n - 1);
52     dst[n - 1] = '\0';
53 }
54
55 /*
56  * symtab_reset: reinicia todos los contadores a 0 y el ambito a "global".
57  * Debe llamarse antes de cada llamada a yyparse() para que un nuevo archivo
58  * comience con tablas limpias.
59  */
60 void symtab_reset(void) {
61     func_count = 0;
62     param_count = 0;
63     var_count = 0;
64     import_count = 0;
65     copy_str(current_scope, sizeof(current_scope), "global");
66 }
67
68 /*
69  * symtab_set_scope: cambia el ambito activo al nombre dado.
70  * Recibe NULL o puntero vacio como sinonimo de "global".
71  */
72 void symtab_set_scope(const char *scope) {
73     copy_str(current_scope, sizeof(current_scope), scope ? scope : "global");

```

```

74 }
75
76 /*
77  * symtab_add_import: registra la sentencia "import <module> [as <alias>]".
78  * Si la tabla esta llena, la entrada se descarta silenciosamente.
79  */
80 void symtab_add_import(const char *module, const char *alias, int line) {
81     if (import_count >= MAX_IMPORTS) {
82         return;
83     }
84     copy_str(imports[import_count].module, sizeof(imports[import_count].module),
85             module);
86     copy_str(imports[import_count].alias, sizeof(imports[import_count].alias),
87             alias);
88     imports[import_count].line = line;
89     import_count++;
90 }
91
92 /*
93  * symtab_add_function: registra la funcion recién definida con 'def'.
94  * Despues de la insercion, actualiza current_scope al nombre de la funcion
95  * para que los parametros que siguen queden asociados a ella.
96  * El campo is_triton_jit indica si el decorador @triton.jit fue detectado
97  * antes del def (gestionado por triton_decorator_pending en parser.y).
98  */
99 void symtab_add_function(const char *name, int line, int arity, int
100 is_triton_jit) {
101     if (func_count >= MAX_FUNCS) {
102         return;
103     }
104     copy_str(funcs[func_count].name, sizeof(funcs[func_count].name), name);
105     funcs[func_count].line = line;
106     funcs[func_count].arity = arity;
107     funcs[func_count].is_triton_jit = is_triton_jit;
108     func_count++;
109     symtab_set_scope(name); /* Nuevas variables perteneceran a esta funcion */
110 }
111
112 /*
113  * symtab_add_param: inserta un parametro formal asociado a la funcion dada.
114  * pos es la posicion 0-based dentro de la lista de parametros.
115  * annot almacena la anotacion de tipo si el parametro tiene la forma "id : expr
116  * ".
117  */
118 void symtab_add_param(const char *func, const char *name, int pos,
119                      const char *annot, int line) {
120     if (param_count >= MAX_PARAMS) {
121         return;
122     }
123     copy_str(params[param_count].func, sizeof(params[param_count].func), func);
124     copy_str(params[param_count].name, sizeof(params[param_count].name), name);
125     copy_str(params[param_count].annot, sizeof(params[param_count].annot), annot);
126     params[param_count].position = pos;
127     params[param_count].line = line;
128     param_count++;
129 }
130
131 /*
132  * symtab_add_variable: registra una variable si no existe ya una entrada con
133  * el mismo nombre y scope. Esto evita entradas duplicadas cuando la misma

```



```

130  * variable se asigna varias veces en el mismo cuerpo de funcion.
131  * scope puede ser NULL, en cuyo caso se usa current_scope.
132  */
133 void symtab_add_variable(const char *name, int line, const char *scope) {
134     int i;
135     const char *sc = scope ? scope : current_scope;
136
137     /* Busqueda lineal: si ya existe la variable en el mismo ambito, no se
        duplica */
138     for (i = 0; i < var_count; ++i) {
139         if (strcmp(vars[i].name, name) == 0 &&
140             strcmp(vars[i].scope, sc) == 0) {
141             return;
142         }
143     }
144     if (var_count >= MAX_VARS) {
145         return;
146     }
147     copy_str(vars[var_count].name, sizeof(vars[var_count].name), name);
148     copy_str(vars[var_count].scope, sizeof(vars[var_count].scope), sc);
149     vars[var_count].line = line;
150     var_count++;
151 }
152
153 /*
154  * symtab_print: imprime por stdout todas las entradas de las cuatro tablas
155  * en formato tabular legible. Llamada por driver.c tras PARSE_OK.
156  * El formato es compatible con la salida esperada por los casos de prueba.
157  */
158 void symtab_print(void) {
159     int i;
160     printf("\n==_PARSER_SYMBOL_TABLE_==\n");
161
162     printf("---_IMPORTS_---\n");
163     for (i = 0; i < import_count; ++i) {
164         printf("%d\tline=%d\tmodule=%s\talias=%s\n",
165             i + 1,
166             imports[i].line,
167             imports[i].module,
168             imports[i].alias[0] ? imports[i].alias : "-");
169     }
170
171     printf("---_FUNCTIONS_---\n");
172     for (i = 0; i < func_count; ++i) {
173         printf("%d\tline=%d\tname=%s\tarity=%d\ttriton_jit=%d\n",
174             i + 1,
175             funcs[i].line,
176             funcs[i].name,
177             funcs[i].arity,
178             funcs[i].is_triton_jit);
179     }
180
181     printf("---_PARAMETERS_---\n");
182     for (i = 0; i < param_count; ++i) {
183         printf("%d\tline=%d\tfunc=%s\tparam=%s\tpos=%d\tannot=%s\n",
184             i + 1,
185             params[i].line,
186             params[i].func,
187             params[i].name,
188             params[i].position,
189             params[i].annot[0] ? params[i].annot : "-");
190     }
191 }

```

```

192     printf("---_VARIABLES_---\n");
193     for (i = 0; i < var_count; ++i) {
194         printf("%d\tline=%d\tscope=%s\tname=%s\n",
195             i + 1,
196             vars[i].line,
197             vars[i].scope,
198             vars[i].name);
199     }
200 }

```

4.4.3. Archivo: src/driver.c

El *driver* es el punto de entrada del ejecutable `triton_parser`. Implementa dos flujos de ejecución:

Modo parser (por defecto) Activa `parser_mode = 1`, llama a `lexer_reset_for_parse()` y `syntab_reset()`, invoca `yyparse()` y verifica la ausencia de tokens sobrantes. Si todo es correcto imprime `PARSE_OK` seguido de las dos tablas de símbolos (parser y léxica).

Modo solo-léxico (-t) Activa `lexer_debug_mode = 1` y consume tokens con un bucle `while(yylex())`, reproduciendo la salida del reporte léxico (casos TC-01 – TC-04).

Código de salida 0 Análisis exitoso.

Código de salida 1 Error sintáctico (`syntax_error_seen`).

Código de salida 2 Error léxico (`lexical_error_seen`).

Listing 4: src/driver.c: punto de entrada del analizador sintáctico.

```

1  /*
2  * Archivo : src/driver.c
3  * Proposito: Punto de entrada del analizador sintactico triton_parser.
4  *           Parsea los argumentos de linea de comandos, abre el archivo
5  *           fuente .tri, ejecuta el scanner en modo depuracion (-t) o
6  *           invoca el parser completo (modo por defecto), e imprime el
7  *           resultado con el codigo de salida correspondiente.
8  *
9  * Modos de operacion:
10 *   Sin -t : modo parser completo. Activa parser_mode, llama yyparse() y,
11 *           si no hay errores, imprime PARSE_OK, la tabla del parser y las
12 *           tablas lexicas.
13 *   Con -t : modo solo-lexico. Recorre todos los tokens con yylex() e imprime
14 *           cada uno junto con las tablas de simbolos al final. Compatible
15 *           con la salida del reporte lexico (casos TC-01 a TC-04).
16 *   Con -d : activa el modo de depuracion interna de yacc (yydebug=1).
17 *
18 * Codigos de salida:
19 *   0 -- analisis exitoso (PARSE_OK o modo -t sin errores lexicos).
20 *   1 -- error sintactico detectado por yyparse() o yyerror().
21 *   2 -- error lexico detectado por el scanner (lexical_error_seen = 1).
22 *
23 * Relacion : Incluye lexer/lexer.h para controlar el scanner, parser/syntab.h
24 *           para inicializar la tabla de simbolos e include/tokens.h para
25 *           compartir constantes. El ejecutable resultante (build/
26 *           triton_parser)
27 *           es invocado por los scripts de prueba en tests/ y scripts/.
28 */
29 #include <stdio.h>

```

```

30 #include <stdlib.h>
31 #include <string.h>
32
33 #include "lexer.h"
34 #include "syntab.h"
35 #include "tokens.h"
36
37 /* Declaraciones externas generadas por Bison al compilar parser.y */
38 extern int yylex(void);
39 extern int yyparse(void);
40 extern int yydebug;          /* Flag de depuracion de yacc (--debug)
    */
41 extern int lexical_error_seen; /* 1 si el scanner encontro un token invalido
    */
42 extern int syntax_error_seen; /* 1 si yyerror() fue llamada durante el parse
    */
43
44 /*
45  * usage: imprime la sinopsis del programa en stderr.
46  * Llamada cuando los argumentos son invalidos o no se provee archivo.
47  */
48 static void usage(const char *prog) {
49     fprintf(stderr, "Uso: %s [-t] [-d] <archivo.tri>\n", prog);
50     fprintf(stderr, "    -t modo tokens (solo lexer, como fase lexica)\n");
51     fprintf(stderr, "    -d depuracion yacc\n");
52 }
53
54 /*
55  * main: flujo principal del driver.
56  *
57  * 1. Parseo de flags: -t activa lexer_debug_mode, -d activa yydebug.
58  * 2. Apertura del archivo .tri.
59  * 3. Si es modo -t: consume tokens con yylex() e imprime tablas lexicas.
60  * 4. Si es modo parser: inicializa el estado (parser_mode, reset lexico y
61  * de syntab), invoca yyparse(), verifica tokens sobrantes, e imprime
62  * PARSE_OK + tablas si el analisis fue exitoso.
63  */
64 int main(int argc, char **argv) {
65     const char *input = NULL;
66     int i;
67     int rc;
68
69     /* Parseo de argumentos de linea de comandos */
70     for (i = 1; i < argc; ++i) {
71         if (strcmp(argv[i], "-t") == 0) {
72             lexer_debug_mode = 1;          /* Modo solo-lexico */
73         } else if (strcmp(argv[i], "-d") == 0) {
74             yydebug = 1;                    /* Traza interna de yacc */
75         } else if (argv[i][0] == '-') {
76             usage(argv[0]);
77             return 1;
78         } else {
79             input = argv[i];                /* Archivo fuente .tri */
80         }
81     }
82
83     if (!input) {
84         usage(argv[0]);
85         return 1;
86     }
87
88     /* Abrir el archivo fuente; yyin es la variable global de Flex */
89     yyin = fopen(input, "r");

```

```

90     if (!yyin) {
91         perror("No se pudo abrir el archivo");
92         return 1;
93     }
94
95     /* -- Modo solo-lexico (-t) -----
96     */
97     if (lexer_debug_mode) {
98         while (yylex() != 0) {
99             /* Consumir todos los tokens; cada llamada imprime en emit_token_*
100             */
101         }
102         print_symbol_tables();
103         fclose(yyin);
104         return lexical_error_seen ? 2 : 0;
105     }
106
107     /* -- Modo parser (por defecto) -----
108     */
109
110     /* Activar modo parser y reiniciar estado del scanner y tabla de simbolos */
111     parser_mode = 1;
112     lexer_reset_for_parse();
113     syntab_reset();
114
115     syntax_error_seen = 0;
116     rc = yyparse(); /* Invocar el parser generado por Bison */
117
118     /* Verificar que no haya tokens sobrantes despues del programa completo *
119     * (p. ej. codigo despues de un dedent final inesperado).
120     */
121     if (rc == 0 && !syntax_error_seen) {
122         int extra;
123         while ((extra = yylex()) != 0) {
124             syntax_error_seen = 1;
125             fprintf(stderr,
126                 "SYNTAX_ERROR: line=%d near='', message='tokens extra tras
127                 fin de programa'\n",
128                 line_no);
129             break;
130         }
131     }
132
133     fclose(yyin);
134
135     /* Prioridad: error lexico > error sintactico > exito */
136     if (lexical_error_seen) {
137         return 2;
138     }
139     if (rc != 0 || syntax_error_seen) {
140         return 1;
141     }
142
143     /* Exito: imprimir PARSE_OK y volcar ambas tablas de simbolos */
144     printf("PARSE_OK: archivo '%s' sintacticamente valido.\n", input);
145     syntab_print(); /* Tabla del parser: funciones, params, vars
146     */
147     print_symbol_tables(); /* Tablas lexicas: IDs, enteros, flotantes, cadenas
148     */
149     return 0;
150 }

```

4.4.4. Archivo: parser/parser.y

Este es el núcleo del analizador sintáctico. El archivo se divide en cuatro secciones yacc:

1. **Preámbulo C** (`%{... %}`): variables globales de contexto (`current_func`, `param_pos_counter`, `triton_decorator_pending`), funciones auxiliares de recuperación de lexemas (`id_text`, `int_text`, ...) y prototipo de `validate_stmt_line_end`.
2. **Declaraciones** (`%union,%token,%type`, precedencias): el `%union` define los dos tipos semánticos (`int i` para índices, `char *s` para lexemas); los tokens numéricos coinciden con el Cuadro 1 del reporte léxico; las directivas `%left/%nonassoc/%right` resuelven conflictos de precedencia y el *dangling-else*.
3. **Reglas de la CFG** (`%%...`): 41 no-terminales y 105 alternativas (P-001 – P-105) documentadas en el Apéndice A. Las acciones semánticas insertan entradas en `syntab` y validan restricciones que la gramática por sí sola no puede expresar (`delim_is`, `validate_stmt_line_end`).
4. **Funciones C** (`%% final`): `validate_stmt_line_end` usa `yypeek()` para detectar múltiples sentencias en la misma línea; `yyerror` formatea los mensajes de error sintáctico en el formato estándar del curso.

Trazabilidad CFG → código. Cada producción de la CFG resumida en la Sección 2.3 se implementa directamente en una regla de `parser.y`. La correspondencia es uno-a-uno: no existe código de manejo sintáctico fuera de este archivo (excepto la generación de `INDENT/DEDENT` en el lexer, documentada en la Sección 2.6).

Listing 5: `parser/parser.y`: especificación yacc completa del analizador sintáctico.

```
1  /*
2  * Archivo : parser/parser.y
3  * Proposito: Especificacion yacc/Bison del analizador sintactico para kernels
4  *           Triton GPU escritos en archivos .tri. Define la gramatica libre de
5  *           contexto (CFG) del subconjunto del lenguaje Python/Triton
6  *           reconocido
7  *           por el compilador del Equipo 11, con acciones semanticas que
8  *           actualizan la tabla de simbolos (syntab) y reportan errores.
9  *
10 * Estructura del archivo (secciones yacc):
11 *   Sec.1 Preambulo C -- includes, variables globales y funciones auxiliares.
12 *   Sec.2 Declaraciones -- %union, %token, %type, precedencias y simbolo
13 *   Sec.3 Reglas       -- producciones de la CFG con acciones semanticas.
14 *   Sec.4 Codigo C     -- implementacion de validate_stmt_line_end y yyerror.
15 *
16 * Relacion : Compilado con Bison para generar y.tab.c y y.tab.h.
17 *           Incluye lexer/lexer.h para acceder a las funciones del scanner,
18 *           parser/syntab.h para la tabla de simbolos e include/tokens.h
19 *           para los IDs de token compartidos.
20 */
21
22 %{
23 #include <stdio.h>
24 #include <stdlib.h>
25 #include <string.h>
26 #include "tokens.h" /* IDs de token compartidos (DOC_*) y MAX_LEXEME_LEN */
27 #include "syntab.h" /* API de la tabla de simbolos del parser */
28 #include "lexer.h"  /* API del scanner: line_no, yypeek, lexeme_by_id, ... */
29
30 */
31
32 /* Funciones generadas por Bison y Flex respectivamente */
33 extern int yylex(void);
```

```

31 extern int yypeek(void);
32 extern int yylineno; /* Contador de lineas de Flex (respaldo; se usa line_no)*/
33
34 /* Declarada al final del archivo; llamada por Bison en caso de error. */
35 void yyerror(const char *msg);
36
37 /* yydebug: si se pone a 1 (flag -d en driver.c), Bison traza las acciones. */
38 int yydebug = 0;
39
40 /* syntax_error_seen: bandera global que se activa en yyerror(); revisada por *
41  * driver.c para devolver codigo de salida 1 en caso de error sintactico. */
42 int syntax_error_seen = 0;
43
44 /* -- Variables de contexto para acciones semanticas ----- */
45
46 /* Nombre de la funcion que se esta analizando actualmente; se establece en
47  * def_stmt al leer el TOKEN_IDENTIFIER del nombre de funcion, y se pasa a
48  * symtab_add_function() y symtab_add_param(). */
49 static char current_func[256] = "";
50
51 /* Contador de posicion de parametros (0-based); se reinicia a 0 en cada def.*/
52 static int param_pos_counter = 0;
53
54 /* Flag que indica si se encontro @triton.jit antes del 'def' actual. *
55  * Se activa en la accion de triton_decorator y se consume en def_stmt. */
56 static int triton_decorator_pending = 0;
57
58 /* parse_ok: se pone a 1 al reducir la regla 'program' sin errores. *
59  * No se usa externamente; el estado de exito lo indica syntax_error_seen. */
60 static int parse_ok = 0;
61
62 /* -- Funciones auxiliares para recuperar lexemas de las tablas ----- */
63
64 /* id_text: retorna el nombre del identificador almacenado en el indice idx. */
65 static const char *id_text(int idx) {
66     return lexeme_by_id(100, idx);
67 }
68
69 /* int_text: retorna el literal entero almacenado en el indice idx. */
70 static const char *int_text(int idx) {
71     return lexeme_by_id(200, idx);
72 }
73
74 /* float_text: retorna el literal flotante almacenado en el indice idx. */
75 static const char *float_text(int idx) {
76     return lexeme_by_id(300, idx);
77 }
78
79 /* str_text: retorna el literal cadena almacenado en el indice idx. */
80 static const char *str_text(int idx) {
81     return lexeme_by_id(400, idx);
82 }
83
84 /*
85  * delim_is: compara el lexema del token actual (s) con el caracter esperado
86  * (ch). Devuelve 1 si coinciden; 0 en caso contrario. Se usa en las acciones
87  * de expr e ident_suffix para detectar el '=' de asignacion que no debe
88  * aparecer dentro de una expresion (solo como TOKEN_ASSIGN).
89  */
90 static int delim_is(const char *s, const char *ch) {
91     return s && ch && strcmp(s, ch) == 0;
92 }
93

```

```

94  /* Prototipo interno; implementacion al final del archivo (Sec.4).
    */
95  static void validate_stmt_line_end(void);
96
97  %}
98
99  /* -- Sec.2: Declaraciones yacc -----
    */
100
101  /* parse.error verbose: Bison genera mensajes de error detallados con el
102   * token inesperado y los tokens esperados en ese punto de la gramatica. */
103  %define parse.error verbose
104
105  /*
106   * %union: define el tipo semantico YYSTYPE. Los tokens y no-terminales
107   * pueden transportar un entero (i, para indices de tabla lexica) o un
108   * puntero a char (s, para lexemas de operadores y delimitadores).
109   */
110  %union {
111      char *s; /* Lexema de TOKEN_OPERATOR, TOKEN_ASSIGN, TOKEN_DELIMITER */
112      int i; /* Indice 1-based en tabla lexica (ID, INT, FLOAT, STRING) */
113  }
114
115  /* -- Declaracion de tokens terminales ----- */
116  /* Los numeros coinciden con los IDs del Cuadro 1 del reporte lexico. */
117  %token <i> TOKEN_IDENTIFIER 100 /* Identificadores validos */
118  %token <i> TOKEN_INTEGER 200 /* Literales enteros */
119  %token <i> TOKEN_FLOAT 300 /* Literales de punto flotante */
120  %token <i> TOKEN_STRING 400 /* Cadenas literales */
121  %token <s> TOKEN_OPERATOR 500 /* Operadores (+, -, **, ==, <=, ...) */
122  /*
123   * %token <s> TOKEN_ASSIGN /* El operador '=' de asignacion */
124   * %token <s> TOKEN_DELIMITER 600 /* Delimitadores: ( ) [ ] { } , . : ; @ -> */
125   * %token <s> TOKEN_KEYWORD 101 /* Palabras reservadas no mapeadas a KW_* */
126   * %token TOKEN_COMMENT 700 /* Comentarios (#...); ignorados en el parser */
127   *
128   * %token TOKEN_NEWLINE 950 /* Salto de linea \n */
129   * %token TOKEN_INDENT 910 /* Apertura de bloque indentado */
130   * %token TOKEN_DEDEDENT 911 /* Cierre de bloque indentado */
131   *
132   * %token KW_DEF KW_RETURN KW_IF KW_ELIF KW_ELSE KW_WHILE KW_FOR KW_IN
133   * %token KW_IMPORT KW_AS KW_PASS KW_NONE KW_TRUE KW_FALSE
134   * %token KW_BREAK KW_CONTINUE KW_AND KW_OR KW_NOT KW_IS
135   *
136   * -- Tipos de no-terminales ----- */
137   * Los no-terminales que producen o propagan valores usan <i> o <s>. */
138  %type <i> expr param_list param_list_opt param unary_expr call_expr opt_expr
139  %type <i> subscript list_expr ident_suffix ident_op
140  %type <s> opt_as import_name
141
142  /* -- Declaraciones de precedencia y asociatividad ----- */
143  /* Las reglas se listan de menor a mayor precedencia (de arriba a abajo).
144   * Resuelven conflictos shift/reduce en expresiones y en la construccion
145   * "dangling else" (KW_IF / KW_ELIF / KW_ELSE). */
146  %left TOKEN_NEWLINE /* Menor precedencia: separa sentencias */
147  %left KW_OR /* Operador logico OR */
148  %left KW_AND /* Operador logico AND */
149  %nonassoc KW_NOT /* Operador unario NOT (no asociativo) */
150  %left TOKEN_OPERATOR /* Operadores aritmeticos/relacionales */
151  %nonassoc KW_ELIF /* Resuelve ambigüedad en cadenas if-elif-else */
152  %nonassoc KW_ELSE /* 'else' se asocia al 'if' mas cercano */
153  %right KW_IF /* Expresion condicional: expr IF cond ELSE expr */

```

```

153
154 /* Simbolo inicial de la gramatica */
155 %start program
156
157 /* -- Sec.3: Reglas de la gramatica (CFG) -----
    */
158 %%
159
160 /*
161  * program: simbolo inicial.
162  * Un programa .tri es una secuencia opcional de newlines, seguida del bloque
163  * superior (imports y defs) y posibles newlines al final.
164  * La accion fija parse_ok = 1 para indicar que la reduccion fue exitosa.
165  */
166 program
167     : opt_newlines top_block opt_newlines
168       { parse_ok = 1; }
169     ;
170
171 /*
172  * opt_newlines: cero o mas saltos de linea consecutivos.
173  * Produccion auxiliar que permite lineas en blanco entre declaraciones y al
174  * inicio/fin del archivo sin generar errores sintacticos.
175  */
176 opt_newlines
177     : /* vacio */
178       | opt_newlines TOKEN_NEWLINE
179     ;
180
181 /*
182  * top_block: secuencia de declaraciones de nivel superior.
183  * Solo se permiten imports y definiciones de funcion ('def') en el nivel 0
184  * de indentacion; sentencias sueltas generarian un error en la practica
185  * porque el lexer emitiria DEDENT antes de ellas.
186  */
187 top_block
188     : /* vacio */
189       | top_block opt_newlines top_line
190     ;
191
192 /*
193  * top_line: una declaracion de nivel superior.
194  * Puede ser un import o un def, con newline opcional al final.
195  */
196 top_line
197     : import_stmt
198       | import_stmt TOKEN_NEWLINE
199       | def_stmt
200       | def_stmt TOKEN_NEWLINE
201     ;
202
203 /*
204  * stmt: cualquier sentencia valida dentro de un bloque.
205  * Cubre todos los tipos de sentencias del subconjunto Triton: imports,
206  * definiciones de funcion anidadas, asignaciones, expresiones, control de
207  * flujo y la sentencia nula 'pass'.
208  */
209 stmt
210     : import_stmt
211       | def_stmt
212       | assign_stmt
213       | ident_stmt
214       | if_stmt

```



```

215 | while_stmt
216 | for_stmt
217 | return_stmt
218 | pass_stmt
219 | expr_stmt
220 ;
221
222 /*
223  * import_stmt: sentencia de importacion "import <modulo> [as <alias>]".
224  * La accion semantica inserta el modulo y su alias opcional en syntab.
225  * Soporta modulos compuestos (triton.language) mediante la regla import_name.
226  */
227 import_stmt
228 : KW_IMPORT import_name opt_as
229 {
230     syntab_add_import($2, $3 ? $3 : "", line_no);
231 }
232 ;
233
234 /*
235  * import_name: nombre del modulo, posiblemente compuesto con puntos.
236  * Se construye concatenando segmentos con '.' para producir "triton.language".
237  * El string resultante se aloja con malloc; la accion libera el parcial
238  * anterior.
239  */
239 import_name
240 : TOKEN_IDENTIFIER
241 {
242     $$ = strdup(id_text($1)); /* Nombre simple: "triton" */
243 }
244 | import_name '.' TOKEN_IDENTIFIER
245 {
246     /* Nombre compuesto: concatena "triton" + "." + "language" */
247     size_t len = strlen($1) + strlen(id_text($3)) + 2;
248     char *buf = (char *)malloc(len);
249     snprintf(buf, len, "%s.%s", $1, id_text($3));
250     free($1); /* Liberar el segmento parcial acumulado anteriormente */
251     $$ = buf;
252 }
253 ;
254
255 /*
256  * opt_as: clausula "as <alias>" opcional.
257  * Produce NULL si no hay alias, o el lexema del identificador alias.
258  */
259 opt_as
260 : /* vacio */ { $$ = NULL; }
261 | KW_AS TOKEN_IDENTIFIER { $$ = (char *)id_text($2); }
262 ;
263
264 /*
265  * triton_decorator: reconoce "@triton.jit" en la linea anterior al def.
266  * Solo activa triton_decorator_pending si la forma es exactamente
267  * @triton.jit; otros decoradores se aceptan pero no marcan el flag.
268  */
269 triton_decorator
270 : '@' TOKEN_IDENTIFIER '.' TOKEN_IDENTIFIER
271 {
272     if (strcmp(id_text($2), "triton") == 0 &&
273         strcmp(id_text($4), "jit") == 0) {
274         triton_decorator_pending = 1;
275     }
276 }

```

```

277     ;
278
279 /*
280  * def_stmt: definicion de funcion con sintaxis Python/Triton.
281  * Flujo de acciones semanticas:
282  *   1. Al leer el nombre (TOKEN_IDENTIFIER): se guarda en current_func y
283  *       se reinicia param_pos_counter para la nueva lista de parametros.
284  *   2. Despues del ')':
285  *       - se registra la funcion en syntab con aridad = numero de params ($3)
286  *       y el flag triton_jit tomado de triton_decorator_pending.
287  *       - se limpia triton_decorator_pending para la siguiente definicion.
288  *   3. suite: cuerpo de la funcion (bloque indentado o sentencia simple).
289  */
290 def_stmt
291 : opt_triton_decorator KW_DEF TOKEN_IDENTIFIER
292 {
293     strncpy(current_func, id_text($3), sizeof(current_func) - 1);
294     current_func[sizeof(current_func) - 1] = '\0';
295     param_pos_counter = 0; /* Reiniciar contador de posicion de
296                             parametros */
297 }
298 '(', opt_newlines param_list_opt opt_newlines ')', ':',
299 {
300     /* Registrar la funcion ya con su aridad (param_list_opt devuelve el
301     count) */
302     syntab_add_function(current_func, line_no, $3,
303         triton_decorator_pending);
304     triton_decorator_pending = 0; /* Consumir el flag del decorador */
305 }
306 suite
307 ;
308
309 /*
310  * opt_triton_decorator: el decorador @triton.jit es opcional.
311  * opt_newlines despues del decorador permite que aparezca en su propia linea.
312  */
313 opt_triton_decorator
314 : /* vacio */
315 | triton_decorator opt_newlines
316 ;
317
318 /*
319  * param_list_opt: lista de parametros formales opcional.
320  * Retorna 0 si la lista esta vacia, o el conteo de parametros si hay al menos
321  * uno.
322  * Este valor es el que def_stmt pasa a syntab_add_function como aridad.
323  */
324 param_list_opt
325 : /* vacio */ { $$ = 0; }
326 | param_list { $$ = $1; }
327 ;
328
329 /*
330  * param_list: uno o mas parametros separados por comas.
331  * La coma puede ir seguida de opt_newlines para permitir parametros en
332  * lineas separadas (estilo Python multi-linea entre parentesis).
333  * Retorna la cuenta acumulada de parametros procesados.
334  */
335 param_list
336 : param
337 { $$ = 1; }
338 | param_list opt_newlines ',', opt_newlines param
339 { $$ = $1 + 1; } /* Incrementar el contador por cada parametro adicional

```

```

336         */
337     ;
338 /*
339  * param: un parametro formal, con o sin anotacion de tipo.
340  * Forma simple: "nombre"
341  * Forma anotada: "nombre : tipo_expr" (p. ej. "n_elements : tl.constexpr")
342  * En ambos casos se inserta el parametro en syntab y se avanza
343     param_pos_counter.
344  * Nota: la anotacion se guarda como cadena vacia (simplificacion; el tipo
345  * exacto requeriria evaluar la expresion, fuera del alcance sintactico).
346  */
347 param
348     : TOKEN_IDENTIFIER
349     {
350         syntab_add_param(current_func, id_text($1), param_pos_counter++, "",
351             line_no);
352         $$ = 1;
353     }
354 | TOKEN_IDENTIFIER ':' opt_newlines expr
355 {
356     syntab_add_param(current_func, id_text($1), param_pos_counter++, "",
357         line_no);
358     $$ = 1;
359 }
360 ;
361 /*
362  * suite: cuerpo de una estructura de control (def, if, while, for).
363  * Hay dos formas:
364  * - simple_stmt: una sola sentencia en la misma linea que el ':'.
365  * - bloque indentado: delimitado por TOKEN_INDENT y TOKEN_DEDENT, con
366  *   cero o mas sentencias (block_body) separadas por newlines opcionales.
367  * Los tokens INDENT/DEDENT son generados por el lexer al detectar cambios
368  * en la columna de inicio de linea (pila de indentacion).
369  */
370 suite
371     : simple_stmt
372     | opt_newlines TOKEN_INDENT block_body TOKEN_DEDENT
373     ;
374 /*
375  * block_body: cuerpo de un bloque indentado.
376  * Puede estar vacio (funcion con solo 'pass') o contener multiples sentencias.
377  */
378 block_body
379     : /* vacio */
380     | block_body opt_newlines block_stmt
381     ;
382 /*
383  * block_stmt: sentencia dentro de un bloque indentado.
384  * La version sin TOKEN_NEWLINE llama a validate_stmt_line_end() para
385  * detectar multiples sentencias ilegales en la misma linea sin separador.
386  */
387 block_stmt
388     : stmt TOKEN_NEWLINE
389     | stmt
390     {
391         validate_stmt_line_end();
392     }
393     ;
394

```

```

395 /*
396  * simple_stmt: sentencia unica en la misma linea que el ':' (suite inline).
397  */
398 simple_stmt
399     : stmt
400     ;
401
402 /*
403  * assign_stmt: asignacion con anotacion de tipo "id : tipo = expr".
404  * Esta forma es especifica del subconjunto Triton que usa anotaciones de tipo
405  * en variables locales (p. ej. "offs_am: tl.tensor = ..."). La variable se
406  * registra en symtab con el ambito de la funcion actual.
407  */
408 assign_stmt
409     : TOKEN_IDENTIFIER ':' expr TOKEN_ASSIGN expr
410     {
411         symtab_add_variable(id_text($1), line_no,
412                             current_func[0] ? current_func : "global");
413     }
414     ;
415
416 /*
417  * ident_stmt: sentencia que comienza con un identificador.
418  * Cubre dos casos:
419  *   - Asignacion simple "id = expr": ident_op retorna 1, se registra en symtab.
420  *   - Expresion de identificador puro (llamada, acceso a atributo): ident_op
421  *     retorna 0, no se registra (no es una nueva variable).
422  */
423 ident_stmt
424     : TOKEN_IDENTIFIER ident_op
425     {
426         if ($2) {
427             symtab_add_variable(id_text($1), line_no,
428                                 current_func[0] ? current_func : "global");
429         }
430     }
431     ;
432
433 /*
434  * ident_op: continuacion de una sentencia que comienza con identificador.
435  *   TOKEN_ASSIGN expr -> asignacion (retorna 1, symtab registra la variable).
436  *   ident_suffix      -> expresion sin asignacion (retorna 0, sin registro).
437  */
438 ident_op
439     : TOKEN_ASSIGN expr { $$ = 1; }
440     | ident_suffix      { $$ = 0; }
441     ;
442
443 /*
444  * if_stmt: sentencia condicional con elif y else opcionales.
445  * La ambigüedad del "dangling else" se resuelve con %nonassoc KW_ELIF y
446  * %nonassoc KW_ELSE en la seccion de precedencias: cada else/elif se asocia
447  * al if/elif mas cercano (comportamiento estandar de Python).
448  */
449 if_stmt
450     : KW_IF expr ':' suite elif_parts else_part
451     ;
452
453 /* elif_parts: cero o mas clausulas "elif expr : suite" encadenadas. */
454 elif_parts
455     : /* vacio */
456     | elif_parts KW_ELIF expr ':' suite
457     ;

```

```

458
459 /* else_part: clausula "else : suite" opcional al final del if. */
460 else_part
461     : /* vacio */
462     | KW_ELSE ':' suite
463     ;
464
465 /* while_stmt: bucle "while expr : suite". */
466 while_stmt
467     : KW_WHILE expr ':' suite
468     ;
469
470 /*
471  * for_stmt: iteracion "for id in expr : suite".
472  * Solo se soporta la variable de iteracion simple (sin tuplas), suficiente
473  * para los patrones "for i in range(...)" comunes en kernels Triton.
474 */
475 for_stmt
476     : KW_FOR TOKEN_IDENTIFIER KW_IN expr ':' suite
477     ;
478
479 /*
480  * return_stmt: retorno de funcion, con o sin valor.
481  * La forma sin expresion corresponde a "return" en funciones void de Triton.
482 */
483 return_stmt
484     : KW_RETURN expr
485     | KW_RETURN
486     ;
487
488 /* pass_stmt: sentencia nula Python; valida en cualquier bloque. */
489 pass_stmt
490     : KW_PASS
491     ;
492
493 /* expr_stmt: una expresion usada como sentencia (p. ej. llamadas a funciones).
494 */
495 expr_stmt
496     : expr
497     ;
498
499 /*
500  * ident_suffix: continuacion posible de un identificador en una expresion.
501  * Maneja todos los sufijos que puede tener un identificador en Triton:
502  *   - Acceso a atributo: ".attr" (member_chain)
503  *   - Indexacion: "[subscript]"
504  *   - Llamada: "(args)"
505  *   - Operacion binaria: "op expr"
506  *   - Identidad: "is expr"
507  *   - Condicional inline: "if cond else alt"
508  *   - Separador de expresiones: TOKEN_DELIMITER expr (p. ej. "," en tuplas)
509  *   - Vacio: el identificador termina aqui
510  *
511  * La accion en TOKEN_OPERATOR verifica que '=' no aparezca como operador
512  * dentro de una expresion (solo es valido como TOKEN_ASSIGN en asignaciones).
513 */
514 ident_suffix
515     : member_chain ident_suffix
516     | '[' subscript_list ']' ident_suffix
517     | '(' opt_newlines arg_list_opt opt_newlines ')' ident_suffix
518     | TOKEN_OPERATOR opt_newlines expr
519     {
520         /* El '=' dentro de una expresion es siempre un error semantico */

```

```

520         if (delim_is($1, "=")) {
521             yyerror("asignacion_invalida_dentro_de_expresion");
522         }
523     }
524 | KW_IS expr
525 | KW_IF expr KW_ELSE expr %prec KW_ELSE /* Expresion condicional ternaria
526     */
527 | TOKEN_DELIMITER opt_newlines expr
528 | %empty
529 ;
530
531 /*
532  * expr: expresion del lenguaje Triton.
533  * Cubre todos los tipos de valor que pueden aparecer en el lado derecho de
534  * una asignacion, en condiciones de control de flujo y en argumentos:
535  *   - Literales: entero, flotante, cadena, None, True, False.
536  *   - Identificador con sufixo (acceso, llamada, indexacion, operacion).
537  *   - Operacion binaria: expr OP expr (precedencias definidas en Sec.2).
538  *   - Identidad: expr is expr (para "dtype is tl.float32").
539  *   - Expresion condicional ternaria: expr if cond else alt.
540  *   - Expresion separada por delimitador (coma en tuplas implícitas).
541  *   - Expresion parentizada: (args).
542  *   - Lista literal: [elementos].
543  *   - Indexacion: expr[subscript].
544  *   - Llamada explicita: call_expr.
545  *   - Negacion/inversion unaria: unary_expr.
546  *
547  * La accion en TOKEN_OPERATOR verifica que '=' no sea usado como operador.
548  */
549 expr
550 : TOKEN_INTEGER { $$ = $1; }
551 | TOKEN_FLOAT { $$ = $1; }
552 | TOKEN_STRING { $$ = $1; }
553 | TOKEN_IDENTIFIER ident_suffix { $$ = $1; }
554 | KW_NONE { $$ = 0; }
555 | KW_TRUE { $$ = 0; }
556 | KW_FALSE { $$ = 0; }
557 | expr TOKEN_OPERATOR opt_newlines expr
558 {
559     if (delim_is($2, "=")) {
560         yyerror("asignacion_invalida_dentro_de_expresion");
561     }
562     $$ = 0;
563 }
564 | expr KW_IS expr { $$ = 0; } /* "dtype is tl.
565     float32" */
566 | expr KW_IF expr KW_ELSE expr %prec KW_ELSE { $$ = 0; } /* Condicional
567     ternario */
568 | expr TOKEN_DELIMITER opt_newlines expr { $$ = 0; } /* Coma entre
569     expresiones */
570 | '(' opt_newlines arg_list_opt opt_newlines ')' { $$ = 0; }
571 | list_expr { $$ = 0; }
572 | expr '[' subscript_list ']' { $$ = 0; }
573 | call_expr { $$ = 0; }
574 | unary_expr { $$ = $1; }
575 ;
576
577 /*
578  * opt_expr: expresion opcional (puede estar vacia).
579  * Usada en los limites de slices: "a[: , :]" donde algun extremo es vacio.
580  */
581 opt_expr
582 : /* vacio */ { $$ = 0; }

```

```

579     | expr          { $$ = $1; }
580     ;
581
582 /* subscript_list: uno o mas subscripts separados por coma (e.g. a[i, j]). */
583 subscript_list
584     : subscript
585     | subscript_list ',' subscript
586     ;
587
588 /*
589  * subscript: un indice dentro de corchetes.
590  * Soporta todas las formas de indexacion de tensores Triton:
591  *   - expr          : indice simple (a[i])
592  *   - None          : insercion de dimension (a[None, :])
593  *   - : opt_expr    : slice desde el inicio (a[:n])
594  *   - opt_expr : opt_expr    : slice basico (a[lo:hi])
595  *   - opt_expr : opt_expr : opt_expr : slice con paso (a[lo:hi:step])
596  */
597 subscript
598     : expr          { $$ = $1; }
599     | KW_NONE       { $$ = 0; }
600     | ':' opt_expr   { $$ = 0; }
601     | opt_expr ':' opt_expr { $$ = 0; }
602     | opt_expr ':' opt_expr ':' opt_expr { $$ = 0; }
603     ;
604
605 /*
606  * unary_expr: expresion unaria prefija (negacion, NOT bit a bit).
607  * Se rechazan explicitamente '*' y '/' como operadores unarios ya que
608  * no tienen semantica definida en el subconjunto Triton del curso.
609  */
610 unary_expr
611     : TOKEN_OPERATOR expr
612     {
613         if (delim_is($1, "*") || delim_is($1, "/")) {
614             yyerror("operador unario no permitido");
615         }
616         $$ = $2;
617     }
618     ;
619
620 /*
621  * list_expr: lista literal entre corchetes "[elem, elem, ...]".
622  * Los elementos se tratan como arg_list_opt (misma regla que los argumentos).
623  */
624 list_expr
625     : '[' opt_newlines arg_list_opt opt_newlines ']'
626     { $$ = 0; }
627     ;
628
629 /*
630  * member_chain: cadena de accesos a atributos o delimitadores.
631  * Cubre patrones como ".method", ".attr.method" y los delimitadores de
632  * acceso con "->" que aparecen en anotaciones de retorno de funcion.
633  */
634 member_chain
635     : '.' opt_newlines TOKEN_IDENTIFIER
636     | member_chain '.' opt_newlines TOKEN_IDENTIFIER
637     | TOKEN_DELIMITER opt_newlines TOKEN_IDENTIFIER
638     | member_chain TOKEN_DELIMITER opt_newlines TOKEN_IDENTIFIER
639     ;
640
641 /*

```

```

642 * call_expr: llamada a funcion o metodo.
643 * Cubre cuatro formas comunes en kernels Triton:
644 *   - f(args)                : llamada simple
645 *   - obj.method(args)       : llamada a metodo (con member_chain)
646 *   - f()                    : llamada sin argumentos
647 *   - call_expr.method(args) : encadenamiento de llamadas
648 */
649 call_expr
650 : TOKEN_IDENTIFIER '(' opt_newlines arg_list_opt opt_newlines ')' { $$ = 0;
651   }
652 | TOKEN_IDENTIFIER member_chain '(' opt_newlines arg_list_opt opt_newlines '
653   ')' { $$ = 0; }
654 | TOKEN_IDENTIFIER '(' opt_newlines ')' { $$ = 0; }
655 | call_expr '.' TOKEN_IDENTIFIER '(' opt_newlines arg_list_opt opt_newlines
656   ')' { $$ = 0; }
657 ;
658
659 /* arg_list_opt: lista de argumentos de una llamada, opcional. */
660 arg_list_opt
661 : /* vacio */
662 | arg_list
663 ;
664
665 /* arg_list: uno o mas argumentos separados por coma. */
666 arg_list
667 : arg
668 | arg_list ',' opt_newlines arg
669 ;
670
671 /*
672 * arg: un argumento de llamada.
673 * Puede ser una expresion posicional o un argumento con nombre "key=valor",
674 * que es el patron habitual en llamadas a tl.load(ptr, mask=mask, ...).
675 */
676 arg
677 : expr
678 | TOKEN_IDENTIFIER TOKEN_ASSIGN opt_newlines expr /* Argumento con nombre
679   */
680 ;
681
682 /* -- Sec.4:Codigo C auxiliar -----
683 */
684 %%
685
686 /*
687 * validate_stmt_line_end: verifica que no haya multiples sentencias en la
688 * misma linea sin separador de newline entre ellas.
689 *
690 * Se llama desde block_stmt cuando una sentencia termina sin TOKEN_NEWLINE.
691 * Usa yypeek() para ver el siguiente token sin consumirlo; si ese token puede
692 * iniciar una sentencia nueva (keyword, literal), emite un error sintactico.
693 *
694 * Tokens que terminan legalmente sin newline: TOKEN_DEDENT, EOF (0),
695 * TOKEN_NEWLINE. Cualquier otro token que pueda iniciar sentencia es invalido.
696 *
697 * Ejemplos detectados: "pass pass", "return 5 6", "x = 5 y = 6".
698 */
699 static void validate_stmt_line_end(void) {
700     int t = yypeek();
701
702     /* Fin de bloque, fin de archivo o newline: todo correcto */
703     if (t == TOKEN_DEDENT || t == 0 || t == TOKEN_NEWLINE) {
704         return;

```



```

700     }
701
702     /* Tokens que pueden iniciar una nueva sentencia en la misma linea */
703     switch (t) {
704     case KW_PASS:
705     case KW_RETURN:
706     case KW_WHILE:
707     case KW_FOR:
708     case KW_IMPORT:
709     case KW_DEF:
710     case KW_ELIF:
711     case KW_ELSE:
712     case TOKEN_INTEGER:
713     case TOKEN_FLOAT:
714     case TOKEN_STRING:
715         yyerror("multiples sentencias en la misma linea");
716         break;
717     default:
718         break;
719     }
720 }
721
722 /*
723  * yyerror: funcion de reporte de errores requerida por Bison.
724  * Activa la bandera global syntax_error_seen y escribe en stderr con el
725  * formato esperado por los scripts de prueba:
726  *   SYNTAX_ERROR: line=<n> near='' message='<msg>'
727  */
728 void yyerror(const char *msg) {
729     syntax_error_seen = 1;
730     fprintf(stderr, "SYNTAX_ERROR: line=%d near='' message='%s'\n", line_no, msg
731 );
732 }
733
734 /*
735  * yyparse_started: funcion de inicializacion del parser (uso interno/pruebas).
736  * Activa parser_mode, reinicia el estado del lexer y limpia symtab.
737  * En produccion este mismo flujo lo ejecuta driver.c antes de llamar yyparse().
738  */
739 int yyparse_started(void) {
740     parser_mode = 1;
741     lexer_reset_for_parse();
742     symtab_reset();
743     return 0;
744 }

```

4.5. Ejemplo de salida del parser (Entregable 5)

Al analizar un kernel mínimo triton_kernel_min.tri:

PARSE_OK: archivo 'triton_kernel_min.tri' sintacticamente valido.

=== PARSER SYMBOL TABLE ===

--- IMPORTS ---

```

1 line=1 module=triton      alias=-
2 line=2 module=triton.language alias=tl

```

--- FUNCTIONS ---

```

1 line=5 name=add_kernel  arity=5  triton_jit=1

```

--- PARAMETERS ---

```

1 line=5 func=add_kernel  param=x_ptr  pos=0  annot=-

```

```

2 line=5 func=add_kernel param=y_ptr pos=1 annot=-
3 line=5 func=add_kernel param=output pos=2 annot=-
4 line=5 func=add_kernel param=n_elements pos=3 annot=-
5 line=5 func=add_kernel param=BLOCK_SIZE pos=4 annot=-
--- VARIABLES ---
1 line=7 scope=add_kernel name=pid
--- PARSER SYMBOL TABLE ---
=== SYMBOL TABLE: IDENTIFIERS ===
1 triton
2 tl
...

```

4.6. Mensajes de error (Entregable 4)

Los mensajes de error están estandarizados en dos formatos, generados respectivamente por el scanner y por `yyperror()`:

```

LEXICAL_ERROR: token_id=<999> line=<n> lexeme="<lex>" (<descripcion>)
SYNTAX_ERROR: line=<n> near="'" message='<descripcion>'

```

Ejemplos reales de la suite de pruebas:

```

LEXICAL_ERROR: token_id=<999> line=3 lexeme="x$" (identificador con caracteres invalidos)
SYNTAX_ERROR: line=5 near="'" message='syntax error, unexpected TOKEN_NEWLINE'
SYNTAX_ERROR: line=3 near="'" message='multiples sentencias en la misma linea'
SYNTAX_ERROR: line=4 near='indent' message='Indentacion inconsistente'

```

4.7. Explicación completa de las secciones de Flex (integración con yacc)

En lenguajes con indentación significativa como Python/Triton, el analizador sintáctico no puede operar de forma aislada: depende de que el analizador léxico realice un trabajo *sintético* avanzado. Al trasladar la gestión de la pila de indentación (INDENT/DEDENT) y el control de múltiples sentencias en una línea al archivo `.l`, el código de Flex pasa a formar parte del diseño sintáctico. Esta subsección explica las **tres secciones** del archivo `lexer/lexico_equipo_11_scanner.l` con foco en su integración con `parser.y`; el scanner completo es la versión integrada del analizador léxico del Equipo 11 [3].

4.7.1. Sección 1 — Declaraciones de Flex (%{...%} y definiciones)

El preámbulo `%{...%}` establece el puente con Bison:

- `#include z.tab.h` hereda los **códigos de token** de Bison (TOKEN_IDENTIFIER, KW_DEF, TOKEN_INDENT, TOKEN_DEDENT, ...) y el tipo YYSTYPE del `%union`; `#include "tokens.h"` aporta los IDs numéricos del Cuadro 1.
- **Estado compartido con el parser** declarado como variables de módulo: `parser_mode`, `lexical_error_seen`, `extern int syntax_error_seen`, la pila `indent_stack[256]` con `indent_depth`, la cola `pending_tokens[]`, los contadores `paren_depth/bracket_depth` (para no emitir DEDENT dentro de paréntesis) y el historial `lex_prev`, `lex_prev2`, `lex_prev_line`, `saw_assign_this_line` usado para detectar múltiples sentencias.
- `extern YYSTYPE yylval` es el canal por el que el scanner entrega valores semánticos (índices de tabla en `yylval.i`, lexemas en `yylval.s`) al parser.

- **Mecanismo clave de integración:** la macro YY_DECL renombra la función generada por Flex a yyflexlex(), y un yylex() escrito a mano la envuelve para **inyectar primero** los INDENT/DEDENT pendientes antes de devolver el token real a Bison.

Listing 6: Declaraciones de integración: yylval compartido y el truco YY_DECL (yyflexlex envuelto por yylex).

```

1 extern YYSTYPE yylval;
2 extern int syntax_error_seen;
3
4 /* Flex genera yyflexlex; el wrapper yylex maneja INDENT/DEDENT pendientes. */
5 #define YY_DECL int yyflexlex(void)
6 int yylex(void);
7 %}

```

Las definiciones de patrones relevantes para el parser son ID ([a-zA-Z][a-zA-Z0-9]*), BAD_ID (identificadores con caracteres prohibidos), WS_COUNT (~[\t]+, indentación al inicio de línea) y NEWLINE (\n+).

4.7.2. Sección 2 — Reglas y patrones (%% ...%%)

Cada regla, en parser_mode, fija yylval y ejecuta return TOKEN_*, entregando el token a yyparse(). Tres reglas impactan directamente al análisis sintáctico:

- WS_COUNT (inicio de línea): calcula el ancho de indentación con indent_width() y indent_line_kind() y llama a handle_indent(), que compara contra la cima de la pila y emite TOKEN_INDENT (sangría mayor) o uno o varios TOKEN_DEDENT (sangría menor) que el parser consume en la regla suite.
- BAD_ID (declarada *antes* que ID para tener prioridad en Flex): emite LEXICAL_ERROR (999) sin fragmentar el lexema, marca lexical_error_seen y **frena** el análisis (código de salida 2, sin PARSE_OK).
- NEWLINE devuelve TOKEN_NEWLINE, terminador de sentencia que el parser usa en top_line y block_stmt.

Listing 7: Regla BAD_ID: error léxico atómico con prioridad sobre ID.

```

1 {BAD_ID} {
2     /* Un solo error lexico por lexema invalido (no se descompone en varios
3        tokens). */
4     fprintf(stderr,
5             "LEXICAL_ERROR: token_id=<%d> line=%d lexeme=\"%s\" (identificador
6             con caracteres invalidos)\n",
7             TOKEN_ERROR, line_no, yytext);
8     lexical_error_seen = 1;
9     at_line_start = 0;
10 }

```

El parser consume los tokens sintéticos de indentación en la regla suite:

Listing 8: Regla suite en parser/parser.y: consumo de INDENT/DEDENT.

```

1     param_pos_counter = 0; /* Reiniciar contador de posicion de
2     parametros */
3 }
4 '(', 'opt_newlines param_list_opt opt_newlines ')', ':'
5 {
6     /* Registrar la funcion ya con su aridad (param_list_opt devuelve el
7     count) */
8 }

```

```

6      symtab_add_function(current_func, line_no, $3,
7          triton_decorator_pending);
8      triton_decorator_pending = 0; /* Consumir el flag del decorador */
9  }
10 suite
11 ;
12
13 /*
14  * opt_triton_decorator: el decorador @triton.jit es opcional.
15  * opt_newlines despues del decorador permite que aparezca en su propia linea.
16  */
17 opt_triton_decorator
18 : /* vacio */
19 | triton_decorator opt_newlines
20 ;
21
22 /*
23  * param_list_opt: lista de parametros formales opcional.
24  * Retorna 0 si la lista esta vacia, o el conteo de parametros si hay al menos
25  * uno.
26  * Este valor es el que def_stmt pasa a symtab_add_function como aridad.
27  */
28 param_list_opt
29 : /* vacio */ { $$ = 0; }
30 | param_list { $$ = $1; }
31 ;
32
33 /*

```

4.7.3. Sección 3 — Código C auxiliar (%% final)

Tras el segundo %% se definen las funciones que permiten al scanner asistir al parser:

- `bol_dedent_token()`: cuando un token inicia en columna 0, con la pila de indentación activa y sin paréntesis/corchetes abiertos, emite el DEDENT de cierre de bloque antes del token real.
- `defer_token(tok)`: guarda el token real (y su `yylval`) para emitirlo *después* del INDENT/DEDENT inyectado; soporta el diferimiento de un token.
- `lexer_multistmt_error(tok)` y `lexer_note_token(tok)`: registran el historial de tokens (omitiendo NEWLINE/INDENT/DEDENT) y detectan patrones de **múltiples sentencias en una línea** (pass pass, return 5 6, x = 5 y = 6), señalando el error que complementa a `validate_stmt_line_end` del parser.

Listing 9: Funciones auxiliares del scanner que asisten al parser: `defer_token`, `lexer_multistmt_error`, `lexer_note_token` y `bol_dedent_token` (`lexer/lexico_equipo_11_scanner.l`).

```

1 static int return_token(int tok, const char *lexeme) {
2     save_lexeme(lexeme);
3     return tok;
4 }
5
6 static void defer_token(int tok) {
7     deferred_tok = tok;
8     deferred_yylval = yylval;
9 }
10
11 static int lexer_multistmt_error(int tok) {

```

```

12     if (!parser_mode || line_no != lex_prev_line) {
13         return 0;
14     }
15     if (tok == KW_PASS && lex_prev == KW_PASS) {
16         return 1;
17     }
18     if (tok == TOKEN_INTEGER && lex_prev == KW_PASS) {
19         return 1;
20     }
21     if (tok == TOKEN_INTEGER && lex_prev == TOKEN_INTEGER && lex_prev2 ==
22         KW_RETURN) {
23         return 1;
24     }
25     if (tok == TOKEN_IDENTIFIER && lex_prev == TOKEN_INTEGER &&
26         saw_assign_this_line) {
27         return 1;
28     }
29     return 0;
30 }
31 static void lexer_note_token(int tok) {
32     if (!parser_mode || tok == 0 || tok == TOKEN_NEWLINE || tok == TOKEN_COMMENT
33         ||
34         tok == TOKEN_INDENT || tok == TOKEN_DEDENT) {
35         return;
36     }
37     lex_prev2 = lex_prev;
38     lex_prev = tok;
39     lex_prev_line = line_no;
40 }
41 static int bol_dedent_token(void) {
42     if (!parser_mode || !at_line_start || line_has_ws || indent_depth <= 1) {
43         return 0;
44     }
45     if (paren_depth > 0 || bracket_depth > 0) {
46         return 0;
47     }
48     return handle_indent(0, INDENT_KIND_UNKNOWN);
49 }

```

5. Verificación y Validación

5.1. Resultados experimentales

Hallazgo principal. El analizador sintáctico `triton_parser` alcanza **100 % (200/200)** en las suites JSONL de validación (`curated_100 + adversarial_100`), **100 % de éxito** en la suite unitaria (12/12) y **82.9 % `PARSE_OK`** en el benchmark TRITONHEAL (58/70). La métrica decisiva del proyecto es JSONL: todos los kernels Triton reales se aceptan y todos los casos adversarios se rechazan correctamente, **sin falsos positivos ni falsos negativos**.

5.2. Resumen cuantitativo

Cuadro 7: Cuadro resumen de resultados (corridas locales reproducibles con `make test`, `make jsonl-test` y `make benchmark`).

Experimento	Éxito	Total	Tasa
Suite unitaria (regresión + inválidos + léxico)	12	12	100 %
Suites JSONL (<code>curated_100</code> + <code>adversarial_100</code>)	200	200	100 %
↪ kernels válidos (<code>curated_100</code>)	100	100	100 %
↪ casos inválidos clasificados (<code>adversarial_100</code>)	100	100	100 %
Benchmark TRITONHEAL (kernels <code>k_*.tri</code>)	58	70	82.9 %
↪ mutaciones sintácticas rechazadas	2	2	100 %
↪ código suelto en columna 0 rechazado	10	10	correcto
Errores léxicos en benchmark	0	70	0 %

5.3. Suites JSONL (make jsonl-test)

Las suites `curated_100.jsonl` y `adversarial_100.jsonl` constituyen el banco de prueba principal del analizador sobre kernels Triton reales e inválidos deliberados. El script `scripts/run_jsonl_tests.sh` ejecuta cada caso, compara el resultado con la etiqueta `should_parse` y genera informes en `results/jsonl/`.

Cuadro 8: Resultados JSONL (métrica principal del proyecto).

Suite	PASS	Total	Descripción
<code>curated_100</code>	100	100	Kernels Triton válidos (deben dar <code>PARSE_OK</code>)
<code>adversarial_100</code>	100	100	Errores léxicos, sintácticos e indentación (deben rechazar)
Total	200	200	Sin falsos positivos ni falsos negativos

Casos adversarios corregidos en la última revisión: `invalid_062` (`pass pass`), `invalid_064` (`return 5 6`), `invalid_065` (`pass 5`), `invalid_073` (`x = 5 y = 6`) y `invalid_092` (dedent súbito a columna 0). Kernel `curated` recuperado: índice 93 con expresión condicional `expr if dtype is tl.float32 else expr`.

5.4. Interpretación de los resultados

Los resultados deben leerse en tres niveles:

1. **Suites JSONL (200/200).** Validan cobertura real sobre 100 kernels Triton compilables y 100 casos de error controlados. Es la evidencia principal de que el parser no rechaza código válido ni acepta inválido en el subconjunto Triton del curso.
2. **Suite unitaria (12/12 PASS).** Valida entregables del curso: acepta `ejemplo_1.tri-ejemplo_3.tri`, kernels mínimos, rechaza cinco archivos inválidos, rechaza `BAD_ID` con `LEXICAL_ERROR` y conserva el modo `-t`.
3. **Benchmark de 70 kernels (58 PARSE_OK).** Mide desempeño sobre el banco TRITONHEAL heredado del proyecto. De 12 rechazos: **2** son mutaciones sintácticas sembradas (`k_0040`, `k_0058`); **10** son archivos con sentencias sueltas en columna 0 tras el `def` (estructura inválida en Python que el parser ahora detecta con `TOKEN_DEDENT`).

Conclusión: la métrica relevante para código Triton correcto es **100/100 en curated_100** y **100/100 en adversarial_100**. El benchmark refleja además la rigurosidad del modelo de indentación Python.

5.5. Gráfica de resultados (Figura 2)

La Figura 2 presenta el desglose del benchmark TRITONHEAL: tasa global, mapa kernel por kernel y comparación con la suite unitaria.

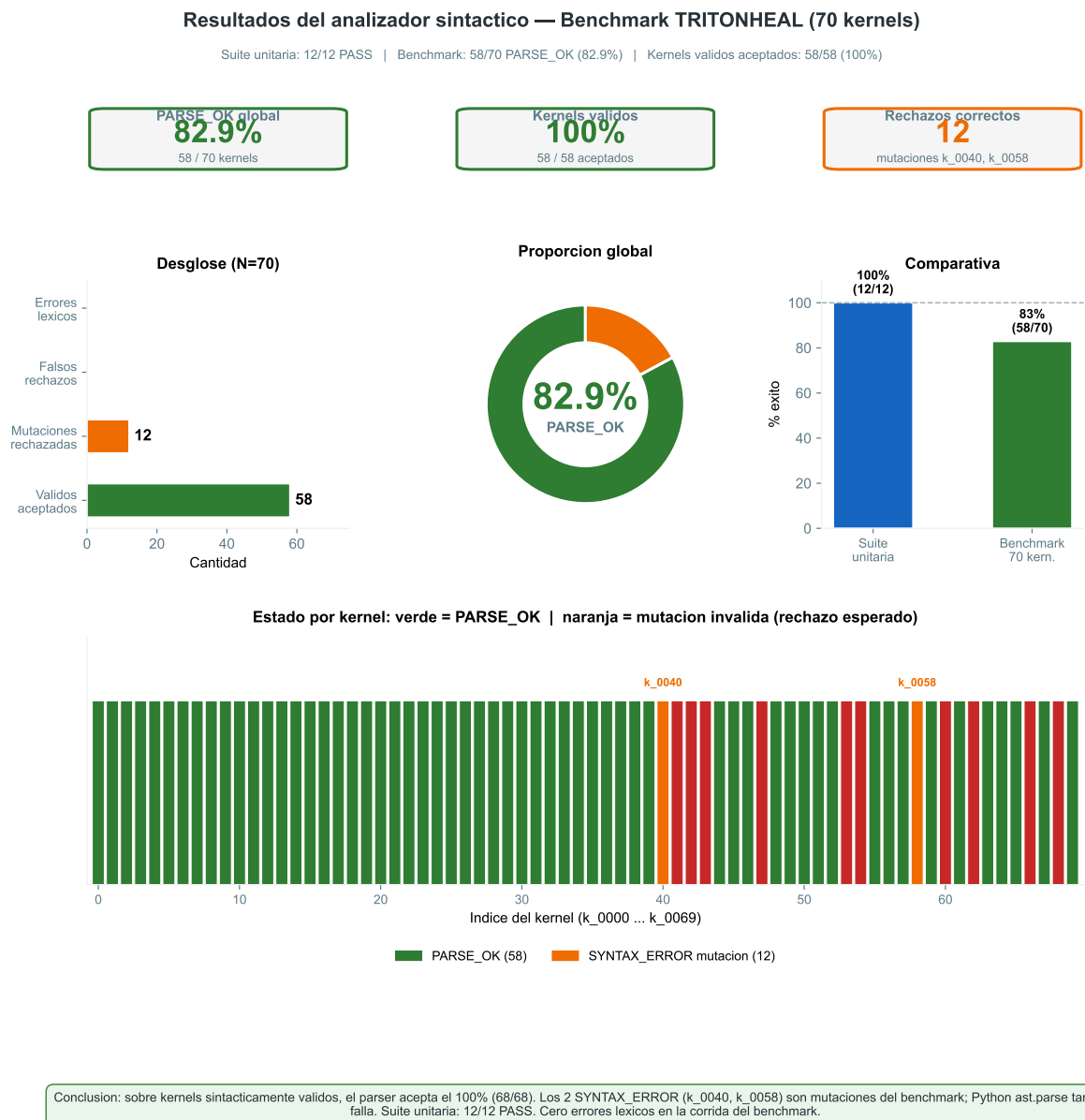


Figura 2: **Resultados del analizador sintáctico (benchmark 70 kernels)**. Verde = PARSE_OK; naranja/rojo = rechazo (mutaciones k_0040/k_0058 o código en columna 0). La validación JSONL 200/200 no se grafica aquí; ver Tabla 8.

5.6. Detalle de rechazos en el benchmark

Cuadro 9: Clasificación de los 12 rechazos del benchmark TRITONHEAL.

Categoría	Kernels	Motivo
Mutaciones inválidas	k_0040, k_0058	Paréntesis
Código en columna 0	k_0041–k_0043, k_0047, k_0053, k_0054, k_0060, k_0062, k_0066, k_0068	ast.pars Sentencia rechazo c

5.7. Modelo de pruebas

Verificación: ¿se construyó el producto correctamente? — compila, ejecuta, mensajes coherentes.

Validación: ¿es el producto correcto? — acepta Triton válido y rechaza inválido.

5.8. Suite unitaria (make test)

Cuadro 10: Casos de prueba del equipo (12 ejecuciones).

ID	Descripción	Resultado
V1–V3	ejemplo_1..3.tri (reporte léxico)	PASS
V4–V5	Kernels Triton mínimos	PASS
I1–I5	Sintaxis inválida deliberada	SYNTAX_ERROR
I6	Identificador con BAD_ID (x\$)	LEXICAL_ERROR
L1–L2	Modo -t tokens	PASS
Total PASS		12 / 12

5.9. Evidencia: salida exitosa (ejemplo_1.tri)

5.10. Evidencia: error sintáctico (def_sin_dos_puntos.tri)

5.11. Evidencia: error léxico BAD_ID (I6)

5.12. Reproducibilidad

Todo el material para reproducir el proyecto en una **PC nueva** está en el repositorio público (sin dependencias externas de otros repos):

- Repositorio: <https://github.com/G4ndhiV/Analizador-sint-ctico>
- Suites JSONL: curated_100.jsonl, adversarial_100.jsonl
- Kernels de prueba: benchmark/kernels/ (70 archivos k_*.tri)
- Fixtures léxicos: tests/fixtures/lexico/
- CFG documentada: report/cfg/triton_cfg.bnf
- Resultados de referencia: results/parser/, results/jsonl/, results/benchmark/
- Informe PDF: deliverables/Informe_Analizador_Sintactico_Equipo11.pdf

5.12.1. Prerrequisitos

Cuadro 11: Software necesario para reproducir compilación, pruebas, gráfica e informe.

Herramienta	Obligatorio	Instalación típica
git	Sí	Clonar el repositorio
cc (GCC/Clang)	Sí	Xcode CLT (macOS) o build-essential (Linux)
Bison	Sí	brew install bison / apt install bison
Flex	Sí	brew install flex / apt install flex
Python 3 + venv	Sí (gráfica)	python3 -m venv .venv; pip install -r requirements.txt
Tectonic o latexmk	Solo para make pdf	brew install tectonic

Verificación rápida: `make check-deps` (objetivo del Makefile raíz).

5.12.2. Pasos desde cero (PC nueva)

1. Clonar:

```
git clone https://github.com/G4ndhiV/Analizador-sint-ctico.git
cd Analizador-sint-ctico
```

2. Comprobar dependencias (opcional): `make check-deps`

3. Compilar: `make build` → ejecutable `build/triton_parser`

4. Suite unitaria: `make test` → **12/12 PASS**; logs en `results/parser/`

5. Suites JSONL: `make jsonl-test` → **200/200 PASS**; informe en `results/jsonl/report.txt`

6. Benchmark 70 kernels: `make benchmark` → `results/benchmark/summary.json`

7. Gráfica: `make figures` → `figures/benchmark_parse_results.pdf`

8. Informe PDF: `make pdf` → `deliverables/Informe_Analizador_Sintactico_Equipo1.pdf`

5.12.3. Comando único (regeneración completa)

```
make clean build test jsonl-test benchmark figures pdf
```

5.12.4. Salidas verificables

- `results/parser/summary.json`: "pass": 12, "total": 12.
- `results/jsonl/summary.json`: curated.pass: 100, adversarial.pass: 100.
- `results/benchmark/summary.json`: parse_ok: 58, total: 70.
- `figures/benchmark_parse_results.pdf` y `figures/benchmark_parse_results.png`.
- PDF del informe en `deliverables/` (Apéndice A: CFG de 105 producciones).

5.12.5. Consulta sin recompilar

Basta con clonar el repositorio y abrir el PDF en `deliverables/` y los archivos `results/js` `onl/summary.json`, `results/parser/summary.json` y `results/benchmark/summary.json`.

5.12.6. Notas por sistema operativo

- **macOS:** instalar `bison` y `flex` con Homebrew; el `Makefile` enlaza la librería Flex de Homebrew.
- **Linux:** usar `bison` y `flex` del sistema; enlace con `-lfl`.
- **BAD_ID:** reglas en `lexer/lexico_equipo_11_scanner.l` (repo léxico: <https://github.com/G4ndhiV/Analizador-lexico>); prueba I6 en `tests/invalid/identificador_caracter_invalido.tri`.

Guía extendida: `README.md` en la raíz del repositorio.

6. Plan de Trabajo

Cuadro 12: Cronograma de la fase sintáctica (Equipo 11).

Fase	Actividad	Entregable
1	Revisión tokens léxicos y kernels TRITONHEAL	Inventario de requisitos
2	Diseño CFG + resolución indentación	Documento de análisis
3	Especificación yacc + symtab	<code>parser.y</code> , <code>symtab.c</code>
4	Integración Flex-yacc + driver	<code>triton_parser</code>
5	Suite de pruebas + benchmark 70	<code>tests/</code> , <code>results/</code>
6	Reporte IEEE-830 + gráfica	PDF final

6.1. Distribución de responsabilidades

- Gramática y yacc: análisis conjunto del equipo, implementación en `parser.y`.
- Lexer integrado: extensión de `lexico_equipo_11_scanner.l`.
- Pruebas y benchmark: scripts `run_tests.sh`, `run_benchmark_kernels.sh`.
- Reporte y figuras: $\text{\LaTeX}$ + `plot_benchmark_results.py`.

A. Gramática libre de contexto completa

Esta sección es la **referencia formal exhaustiva** del analizador: cada no terminal, cada alternativa de producción y cada terminal declarado en `parser/parser.y` quedan documentados sin omisiones. El listado numerado P-001...P-105 en el Archivo 10 corresponde a las 105 alternativas de los 41 no terminales de la gramática yacc.

A.1. Convenciones y notación

- **Forma:** `NT ->cuerpo`; varias líneas con el mismo NT son alternativas (OR).
- ε / `epsilon`: producción vacía (`/* empty */` en Bison).

- **Prefijo P-NNN**: identificador de trazabilidad en `triton_cfg.bnf`; no es un símbolo del lenguaje.
- **Terminales léxicos**: se escriben como en yacc (`TOKEN_*`, `KW_*`) o como literales entre comillas simples.
- **Acciones semánticas**: no forman parte de la CFG; se listan en la Subsección A.6 solo como notas de implementación.

A.2. Símbolo inicial y conteo

Cuadro 13: Resumen formal de la gramática implementada.

Concepto	Valor
Símbolo inicial (<code>%start</code>)	<code>program</code>
No terminales con producciones	41
Alternativas de producción (total)	105
Archivo de implementación	<code>parser/parser.y</code>
Archivo BNF documentado	<code>report/cfg/triton_cfg.bnf</code>

A.3. Terminales del analizador

A.3.1. Terminales que aparecen en producciones

Cuadro 14: Terminales usados en al menos una producción de la CFG.

Símbolo yacc	ID	Rol en kernels .tri
<code>TOKEN_IDENTIFIER</code>	100	Nombres, módulos, funciones
<code>TOKEN_INTEGER</code>	200	Literales enteros
<code>TOKEN_FLOAT</code>	300	Literales flotantes
<code>TOKEN_STRING</code>	400	Cadenas
<code>TOKEN_OPERATOR</code>	500	Operadores binarios/unarios, = en asignación
<code>TOKEN_DELIMITER</code>	600	Delimitadores en expresiones y <code>member_chain</code>
<code>TOKEN_NEWLINE</code>	950	Fin de línea
<code>TOKEN_INDENT</code>	910	Inicio de bloque (desde <code>WS_COUNT</code>)
<code>TOKEN_DEDENT</code>	911	Fin de bloque
<code>KW_IMPORT</code> , <code>KW_AS</code>	101	<code>import/as</code>
<code>KW_DEF</code>	101	Definición de kernel
<code>KW_IF</code> , <code>KW_ELIF</code> , <code>KW_ELSE</code>	101	Condicionales
<code>KW_WHILE</code> , <code>KW_FOR</code> , <code>KW_IN</code>	101	Bucles
<code>KW_RETURN</code> , <code>KW_PASS</code>	101	Control de flujo
<code>KW_IS</code>	101	Comparación de identidad (<code>is</code>)
<code>KW_NONE</code> , <code>KW_TRUE</code> , <code>KW_FALSE</code>	101	Constantes
<code>TOKEN_ASSIGN</code>	—	Asignación (=) separada de operadores
<code>'@'</code> , <code>'.'</code> , <code>'.'</code> , <code>'('</code> , <code>','</code> , <code>')</code>	—	Decorador, acceso, agrupación
<code>'['</code> , <code>']'</code> , <code>':'</code> , <code>','</code> , <code>','</code>	—	Subíndices, slices, listas

A.3.2. Terminales declarados y no usados en reglas

En `parser.y` (líneas 53–59) se declaran además `TOKEN_COMMENT`, `TOKEN_KEYWORD` y `KW_BREAK`, `KW_CONTINUE`, `KW_AND`, `KW_OR`, `KW_NOT`. **Ninguna producción** de la CFG los referencia: el lexer no los entrega al parser en modo normal (`COMMENT` se descarta; las palabras no modeladas en `stmt/expr` producirían error sintáctico si se usaran).

A.4. Precedencia y asociatividad (yacc)

Cuadro 15: Directivas `%left` / `%nonassoc` / `%right` en `parser.y` (L71–78).

Nivel	Asoc.	Terminales
1 (más baja)	izquierda	TOKEN_NEWLINE
2	izquierda	KW_OR
3	izquierda	KW_AND
4	no asoc.	KW_NOT
5	izquierda	TOKEN_OPERATOR, TOKEN_DELIMITER (en <code>expr</code>)
6	no asoc.	KW_ELIF
7	no asoc.	KW_ELSE
8 (más alta)	derecha	KW_IF (expresión condicional)

Nota: la jerarquía de `expr` está **colapsada** en un solo no terminal; la precedencia entre operadores concretos (+, *, etc.) depende del orden de lectura y de estas declaraciones, no de niveles `expr_add`, `expr_mul`, etc.

A.5. Inventario de no terminales (41)

Cuadro 16: Catálogo de no terminales: número de alternativas e identificadores P- en `triton_cfg.bnf`.

No terminal	Alt.	IDs P-
<code>program</code>	1	001
<code>opt_newlines</code>	2	002–003
<code>top_block</code>	2	004–005
<code>top_line</code>	4	006–009
<code>suite</code>	2	010–011
<code>block_body</code>	2	012–013
<code>block_stmt</code>	2	014–015
<code>simple_stmt</code>	1	016
<code>stmt</code>	10	017–026
<code>import_stmt</code>	1	027
<code>import_name</code>	2	028–029
<code>opt_as</code>	2	030–031
<code>triton_decorator</code>	1	032
<code>opt_triton_decorator</code>	2	033–034
<code>def_stmt</code>	1	035
<code>assign_stmt</code>	1	036
<code>ident_stmt</code>	1	037
<code>ident_op</code>	2	038–039
<code>if_stmt</code>	1	040
<code>elif_parts</code>	2	041–042
<code>else_part</code>	2	043–044
<code>while_stmt</code>	1	045
<code>for_stmt</code>	1	046
<code>return_stmt</code>	2	047–048
<code>pass_stmt</code>	1	049

(continuación)

No terminal	Alt.	IDs P-
expr_stmt	1	050
param_list_opt	2	051–052
param_list	2	053–054
param	2	055–056
ident_suffix	8	057–064
expr	16	065–080
opt_expr	2	081–082
subscript_list	2	083–084
subscript	5	085–089
unary_expr	1	090
list_expr	1	091
member_chain	4	092–095
call_expr	4	096–099
arg_list_opt	2	100–101
arg_list	2	102–103
arg	2	104–105
Total (41 no terminales)	105	001–105

A.6. Notas semánticas ligadas a la CFG (fuera de producciones)

Cuadro 17: Acciones en C asociadas a reglas seleccionadas (no son parte de la CFG).

Regla	Efecto
program	parse_ok := 1 al reducir.
import_stmt	syntab_add_import.
triton_decorator	Activa bandera si lexemas son triton.jit.
def_stmt	Tras ID: current_func; tras ')' ' ': syntab_add_function.
param	syntab_add_param por cada parámetro.
assign_stmt	Variable en syntab solo si TOKEN_OPERATOR es '='.

A.7. Desglose detallado: no terminal expr (16 alternativas, P-065–080)

Todas las formas de expresión admitidas por el parser:

1. TOKEN_INTEGER (P-065)
2. TOKEN_FLOAT (P-066)
3. TOKEN_STRING (P-067)
4. TOKEN_IDENTIFIER ident_suffix (identificador con sufijo opcional, P-068)
5. KW_NONE (P-069)
6. KW_TRUE (P-070)
7. KW_FALSE (P-071)
8. expr TOKEN_OPERATOR expr (binario, P-072)
9. expr KW_IS expr (identidad is, P-073)

10. `expr KW_IF expr KW_ELSE expr` (expresión condicional, P-074)
11. `expr TOKEN_DELIMITER expr` (binario con delimitador léxico, P-075)
12. `'(' arg_list_opt ')'` (tupla / agrupación, P-076)
13. `list_expr` (literal de lista [...], P-077)
14. `expr '[' subscript_list ']'` (indexación, P-078)
15. `call_expr` (P-079)
16. `unary_expr` (P-080)

A.8. Desglose detallado: subscript (5 alternativas, P-085–089)

Soporta índices y slices estilo Python/Triton: `expr` (P-085), `KW_NONE` (P-086), `':' opt_expr` (P-087), `opt_expr ':' opt_expr` (P-088) y `opt_expr ':' opt_expr ':' opt_expr` (P-089), con `opt_expr` opcional en cada posición del slice.

A.9. Desglose detallado: block_stmt (2 alternativas, P-014–015)

Permite una sentencia terminada en `NEWLINE` (`stmt TOKEN_NEWLINE`, P-014) o una sentencia final sin `NEWLINE` (`stmt`, P-015, validada por `validate_stmt_line_end`). Las líneas en blanco dentro del bloque las absorbe `opt_newlines` en `block_body`.

A.10. Listado completo de producciones (P-001 a P-105)

Listing 10: CFG completa numerada: 41 no terminales, 105 alternativas (`report/cfg/triton_cfg.bnf`).

```

1  % =====
2  % CFG COMPLETA - Analizador sintactico Triton (.tri) - Equipo 11
3  % Fuente canonica de implementacion: parser/parser.y (reglas entre %% ... %%)
4  % Simbolo inicial: program
5  % No terminales: 41 / Producciones (alternativas): 105 / Ultima revision: 2026-06
6  % Notacion: NT -> cuerpo      (una alternativa por linea con prefijo P-NNN)
7  % epsilon = produccion vacia (/* empty */ en yacc)
8  % =====
9
10 % -----
11 % 1. PROGRAMA Y ESTRUCTURA DE BLOQUES
12 % -----
13
14 P-001  program          -> opt_newlines top_block opt_newlines
15
16 P-002  opt_newlines     -> epsilon
17 P-003  opt_newlines     -> opt_newlines TOKEN_NEWLINE
18
19 P-004  top_block        -> epsilon
20 P-005  top_block        -> top_block opt_newlines top_line
21
22 P-006  top_line         -> import_stmt
23 P-007  top_line         -> import_stmt TOKEN_NEWLINE
24 P-008  top_line         -> def_stmt
25 P-009  top_line         -> def_stmt TOKEN_NEWLINE
26
27 P-010  suite            -> simple_stmt
28 P-011  suite            -> opt_newlines TOKEN_INDENT block_body TOKEN_DEDENT
29
30 P-012  block_body       -> epsilon
31 P-013  block_body       -> block_body opt_newlines block_stmt
32
33 P-014  block_stmt       -> stmt TOKEN_NEWLINE
34 P-015  block_stmt       -> stmt

```

```

35
36 P-016  simple_stmt      -> stmt
37
38 % -----
39 % 2. SENTENCIAS (stmt y derivados)
40 % -----
41
42 P-017  stmt             -> import_stmt
43 P-018  stmt             -> def_stmt
44 P-019  stmt             -> assign_stmt
45 P-020  stmt             -> ident_stmt
46 P-021  stmt             -> if_stmt
47 P-022  stmt             -> while_stmt
48 P-023  stmt             -> for_stmt
49 P-024  stmt             -> return_stmt
50 P-025  stmt             -> pass_stmt
51 P-026  stmt             -> expr_stmt
52
53 P-027  import_stmt      -> KW_IMPORT import_name opt_as
54
55 P-028  import_name      -> TOKEN_IDENTIFIER
56 P-029  import_name      -> import_name '.' TOKEN_IDENTIFIER
57
58 P-030  opt_as            -> epsilon
59 P-031  opt_as            -> KW_AS TOKEN_IDENTIFIER
60
61 P-032  triton_decorator -> '@' TOKEN_IDENTIFIER '.' TOKEN_IDENTIFIER
62
63 P-033  opt_triton_decorator -> epsilon
64 P-034  opt_triton_decorator -> triton_decorator opt_newlines
65
66 P-035  def_stmt         -> opt_triton_decorator KW_DEF TOKEN_IDENTIFIER '('
    param_list_opt ')' ':' suite
67
68 P-036  assign_stmt      -> TOKEN_IDENTIFIER ':' expr TOKEN_ASSIGN expr
69
70 P-037  ident_stmt       -> TOKEN_IDENTIFIER ident_op
71
72 P-038  ident_op         -> TOKEN_ASSIGN expr
73 P-039  ident_op         -> ident_suffix
74
75 P-040  if_stmt          -> KW_IF expr ':' suite elif_parts else_part
76
77 P-041  elif_parts       -> epsilon
78 P-042  elif_parts       -> elif_parts KW_ELIF expr ':' suite
79
80 P-043  else_part        -> epsilon
81 P-044  else_part        -> KW_ELSE ':' suite
82
83 P-045  while_stmt       -> KW_WHILE expr ':' suite
84
85 P-046  for_stmt         -> KW_FOR TOKEN_IDENTIFIER KW_IN expr ':' suite
86
87 P-047  return_stmt      -> KW_RETURN expr
88 P-048  return_stmt      -> KW_RETURN
89
90 P-049  pass_stmt        -> KW_PASS
91
92 P-050  expr_stmt        -> expr
93
94 % -----
95 % 3. PARAMETROS DE FUNCION
96 % -----
97
98 P-051  param_list_opt   -> epsilon
99 P-052  param_list_opt   -> param_list
100
101 P-053  param_list       -> param
102 P-054  param_list       -> param_list opt_newlines ',' opt_newlines param
103
104 P-055  param            -> TOKEN_IDENTIFIER
105 P-056  param            -> TOKEN_IDENTIFIER ':' opt_newlines expr
106

```



```

107 % -----
108 % 4. SUFIJOS DE IDENTIFICADOR Y EXPRESIONES
109 % -----
110
111 P-057 ident_suffix      -> member_chain ident_suffix
112 P-058 ident_suffix      -> '[' subscript_list ']' ident_suffix
113 P-059 ident_suffix      -> '(' opt_newlines arg_list_opt opt_newlines ')' ident_suffix
114 P-060 ident_suffix      -> TOKEN_OPERATOR opt_newlines expr
115 P-061 ident_suffix      -> KW_IS expr
116 P-062 ident_suffix      -> KW_IF expr KW_ELSE expr
117 P-063 ident_suffix      -> TOKEN_DELIMITER opt_newlines expr
118 P-064 ident_suffix      -> epsilon
119
120 P-065 expr              -> TOKEN_INTEGER
121 P-066 expr              -> TOKEN_FLOAT
122 P-067 expr              -> TOKEN_STRING
123 P-068 expr              -> TOKEN_IDENTIFIER ident_suffix
124 P-069 expr              -> KW_NONE
125 P-070 expr              -> KW_TRUE
126 P-071 expr              -> KW_FALSE
127 P-072 expr              -> expr TOKEN_OPERATOR opt_newlines expr
128 P-073 expr              -> expr KW_IS expr
129 P-074 expr              -> expr KW_IF expr KW_ELSE expr
130 P-075 expr              -> expr TOKEN_DELIMITER opt_newlines expr
131 P-076 expr              -> '(' opt_newlines arg_list_opt opt_newlines ')'
132 P-077 expr              -> list_expr
133 P-078 expr              -> expr '[' subscript_list ']'
134 P-079 expr              -> call_expr
135 P-080 expr              -> unary_expr
136
137 P-081 opt_expr          -> epsilon
138 P-082 opt_expr          -> expr
139
140 P-083 subscript_list    -> subscript
141 P-084 subscript_list    -> subscript_list ',' subscript
142
143 P-085 subscript         -> expr
144 P-086 subscript         -> KW_NONE
145 P-087 subscript         -> ':' opt_expr
146 P-088 subscript         -> opt_expr ':' opt_expr
147 P-089 subscript         -> opt_expr ':' opt_expr ':' opt_expr
148
149 P-090 unary_expr        -> TOKEN_OPERATOR expr
150
151 P-091 list_expr         -> '[' opt_newlines arg_list_opt opt_newlines ']'
152
153 P-092 member_chain      -> '.' opt_newlines TOKEN_IDENTIFIER
154 P-093 member_chain      -> member_chain '.' opt_newlines TOKEN_IDENTIFIER
155 P-094 member_chain      -> TOKEN_DELIMITER opt_newlines TOKEN_IDENTIFIER
156 P-095 member_chain      -> member_chain TOKEN_DELIMITER opt_newlines TOKEN_IDENTIFIER
157
158 P-096 call_expr         -> TOKEN_IDENTIFIER '(' opt_newlines arg_list_opt opt_newlines
159                        ')',
160 P-097 call_expr         -> TOKEN_IDENTIFIER member_chain '(' opt_newlines arg_list_opt
161                        opt_newlines ')',
162 P-098 call_expr         -> TOKEN_IDENTIFIER '(' opt_newlines ')'
163 P-099 call_expr         -> call_expr '.' TOKEN_IDENTIFIER '(' opt_newlines arg_list_opt
164                        opt_newlines ')',
165
166 P-100 arg_list_opt      -> epsilon
167 P-101 arg_list_opt      -> arg_list
168
169 P-102 arg_list          -> arg
170 P-103 arg_list          -> arg_list ',' opt_newlines arg
171
172 P-104 arg               -> expr
173 P-105 arg               -> TOKEN_IDENTIFIER TOKEN_ASSIGN opt_newlines expr
174
175 % =====
176 % TERMINALES ADICIONALES
177 % =====
178 % TOKEN_ASSIGN - asignacion (=) separada de TOKEN_OPERATOR
179 % KW_IS - operador is (comparacion de identidad)

```

```

177 % Acciones semanticas (fuera de CFG):
178 %   validate_stmt_line_end() en block_stmt sin NEWLINE
179 %   lexer_multistmt_error() en yylex_wrapper
180 %   bol_dedent_token() para DEDENT en columna 0

```

A.11. Implementación yacc (referencia cruzada)

Las reglas yacc ejecutables (con sus acciones semánticas) que implementan estas producciones se listan completas en la Sección 3.3 y el printout íntegro de `parser.y` de la Sección de Implementación (Listado 5); no se reproducen aquí para evitar duplicación.

A.12. Trazabilidad y mantenimiento

Cualquier modificación en `parser/parser.y` exige actualizar `triton_cfg.bnf` (renumerar P-*, tabla 16 y conteo 91) y recompilar el PDF. La Sección 2 del análisis conserva un **resumen**; este apéndice es la definición **completa**.

B. Referencias

1. R. J. Castelló, “Chapter 3: Syntax Analysis, Part I — Grammar Processing,” *Module #3: Compilers* (TC3002B, Advanced Applications Development), notas de curso, ITESM, s. f.
2. R. J. Castelló, “Chapter 2: Lexical Analysis,” *Module #3: Compilers* (TC3002B), notas de curso, ITESM, s. f.
3. Equipo 11, *Analizador Léxico — Triton GPU Kernel*, reporte TC3002B, entrega mayo 2026; implementación Flex en <https://github.com/G4ndhiV/Analizador-lexico> (`lexico_equipo_11_scanner.l`, reglas `BAD_ID`).
4. Salvador Hinojosa, *Evaluation Metrics Syntax Analyzer GDL Triton*, TC3002B, documento de evaluación, 2026.
5. OpenAI, *Triton: Open-Source GPU Programming*, [en línea]; disponible en <https://triton-lang.org>; Internet.
6. A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Addison-Wesley, 2007.
7. Proyecto TRITONHEAL, benchmark de 70 kernels `k_*.tri`, repositorio de referencia del Equipo 11, 2026.